

UNIVERSIDAD DE GRANADA

DOCTORAL THESIS

**A framework for data-driven Predictive
Maintenance of industrial equipment
using Intelligent Agents**

Author:

Huy Anh LE

Supervisor:

Dr. Juan Antonio HOLGADO

*A thesis submitted in fulfillment of the requirements
for the degree of Doctor of Philosophy
in the*

**Sistemas Concurrentes
Escuela de Doctorado de Ciencias, Tecnologías e Ingenierías**

May 3, 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objective	1
1.2.1	General objective	1
1.2.2	Specific objectives	1
1.3	Thesis structure	2
2	Problem Description: the Predictive Maintenance problem	3
2.1	Main Section 1	3
2.1.1	Subsection 1	3
2.1.2	Subsection 2	3
2.2	Main Section 2	3
3	Deep Learning for the Predictive Maintenance problem	5
3.1	Predictive Maintenance for Industrial Machines	5
3.1.1	Classification of Maintenance	5
3.1.2	Three levels of Predictive Maintenance	6
3.1.3	Industrial Machine and Machine Data	8
3.1.4	Data-driven approach	8
3.1.5	Examples of Predictive Maintenance application	10
3.2	Deep Learning for the Predictive Maintenance problem	11
3.2.1	Concept of Deep Learning	12
3.2.2	Supervised, Unsupervised and Semi-Supervised Learning	12
3.2.3	Fundamental concepts of Deep Learning	13
3.2.4	Why Deep Learning is preferred for Predictive Maintenance?	19
3.3	Semi-Supervised Learning for Anomaly Detection	22
3.3.1	Supervised learning methods for Predictive Maintenance	22
3.3.2	The need of Semi-Supervised learning	23
3.3.3	Semi-supervised Anomaly Detection	24
3.3.4	Regression models for Anomaly Detection	27
3.4	AutoEncoder for Anomaly Detection	29
3.4.1	Anomaly Detection based on Data Reconstruction Error	29
3.4.2	Introduction of AutoEncoder	30
3.4.3	AutoEncoder-based Anomaly Detection method	35
3.4.4	Referenced studies of Anomaly Detection using AutoEncoder	41
3.4.5	Conclusion	45
4	Proposed Solution: description of the proposed solution	47
4.1	Main Section 1	47
4.1.1	Subsection 1	47
4.1.2	Subsection 2	47
4.2	Main Section 2	47

5	Implemented Solution: detailed implementation of the proposed solution	49
5.1	Main Section 1	49
5.1.1	Subsection 1	49
5.1.2	Subsection 2	49
5.2	Main Section 2	49
6	Case Study 1	51
6.1	Main Section 1	51
6.1.1	Subsection 1	51
6.1.2	Subsection 2	51
6.2	Main Section 2	51
7	Case Study 2	53
7.1	Main Section 1	53
7.1.1	Subsection 1	53
7.1.2	Subsection 2	53
7.2	Main Section 2	53
8	Conclusions and Future Works	55
8.1	Main Section 1	55
8.1.1	Subsection 1	55
8.1.2	Subsection 2	55
8.2	Main Section 2	55
	Bibliography	57

Chapter 1

Introduction

1.1 Motivation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

1.2 Objective

1.2.1 General objective

The general objective of this project is to study related state-of-the-art technologies and then apply them to design, develop and evaluate an Intelligent System which supports Predictive Maintenance of Industrial Machines. This work can be classified as a Practical Application Work (“Trabajo de Aplicación Práctica”)

This intelligent system should be a concrete software and must work with at least one case study.

The data-driven approach for development of the intelligent Predictive Maintenance system should be general and scalable. During the development of this work, these questions can be answered:

- Can industrial machine failures be predicted before they happen using historical and new machine data?
- Is there a general and scalable approach of data-driven predictive maintenance for different types of industrial machine?
- Can the intelligent Predictive Maintenance solution be deployed and used effectively in real life as a software, but not as machine learning models created and used by data scientists?

Because this work focuses on industrial environments, its primary scope of maintenance is Industrial Machines, although its results can also be applied for home, transportation, IT... devices.

1.2.2 Specific objectives

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus

venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

1.3 Thesis structure

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 2

Problem Description: the Predictive Maintenance problem

2.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

2.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

2.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

2.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi

tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 3

Deep Learning for the Predictive Maintenance problem

This chapter presents the review of studies related to Industrial Maintenance problem, especially Predictive Maintenance. Content of this chapter includes the classification of industrial maintenance, the prediction level of each maintenance strategy, the data-driven Predictive Maintenance approach, the reason why Deep Learning is preferred, fundamental of Deep Learning, the comparison between supervised learning and semi-supervised learning for Predictive Maintenance, the concept of Anomaly Detection, introduction of AutoEncoder and the Anomaly Detection method using AutoEncoder. Through this state-of-the-art review, the AutoEncoder-based Anomaly Detection method is chosen for implementation of predictive models of the intelligent data-driven Predictive Maintenance system.

3.1 Predictive Maintenance for Industrial Machines

3.1.1 Classification of Maintenance

As Susto et al. present in their article, maintenance of equipment or systems can be classified into three primary types: Corrective Maintenance, Preventive Maintenance and Predictive Maintenance (PdM) (Susto et al., 2014). Corrective Maintenance (also referred to as Run-to-Failure) is the way in which maintenance interventions are performed only after occurrence of failures (Fig. 3.1). This is the simplest approach and also the least effective one. Preventive Maintenance (also referred to as Scheduled Maintenance) is the maintenance policy where maintenance actions are carried out according to a scheduled plan based on time or process iteration (Fig. 3.2). This approach can prevent most of failures, but it is usually heavy with unnecessary corrective actions, inefficient use of resources and increased operating costs. When the organization cannot keep the heavy maintenance schedule, it easily becomes Run-to-Failure. Predictive Maintenance (also referred to as Prognosis) is the approach where maintenance is performed based on an estimation of the health status of the equipment or system (Fig. 3.3). A Predictive Maintenance system is a software which can detect in advance potential failures for timely interventions. The cores of a Predictive Maintenance system are predictive models built on historical data, predictive algorithms and related technologies.

In summary, Corrective Maintenance is too late, while Preventive Maintenance can be too early. Predictive Maintenance is expected to be on time, neither early nor late. Therefore, Predictive Maintenance is the objective and also the new standard of Smart Manufacturing.

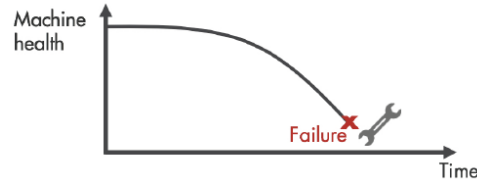


FIG. 3.1: Corrective Maintenance (source: MathWorks)

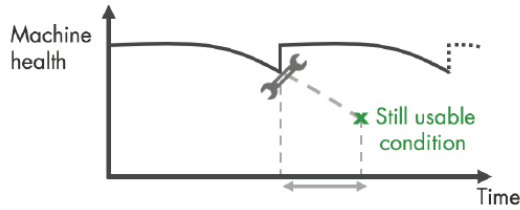


FIG. 3.2: Preventive Maintenance (source: MathWorks)

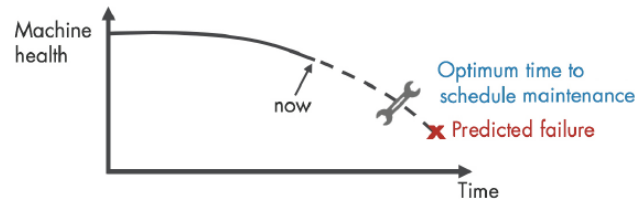


FIG. 3.3: Predictive Maintenance (source: MathWorks)

3.1.2 Three levels of Predictive Maintenance

In terms of prediction level, Predictive Maintenance can be classified to 3 levels: Anomaly Detection, Diagnosis, and Prognosis (Khan and Yairi, 2018).

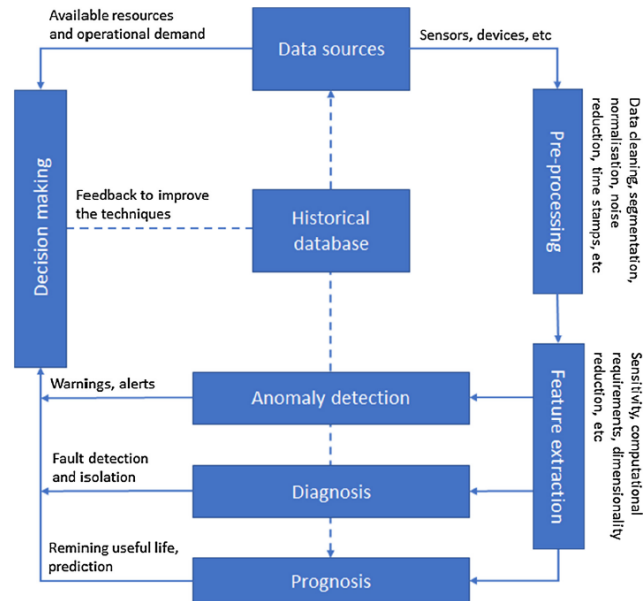


FIG. 3.4: Three levels of Predictive Maintenance (Khan and Yairi, 2018)

The first level of Predictive Maintenance is Anomaly Detection which can detect if there are anomalies. Depending on the level of anomaly, warning or alarm can be raised. The second level is Diagnosis (or Fault Identification and Isolation). This level can identify what the fault type is and where it comes from (which component in which operating cycle). The highest level of Predictive Maintenance is Prognosis. Prognosis is a process of estimation of the Remaining Useful Life (RUL) of machines in order to predict how long the machine still works in acceptable good state before maintenance must be done. Fig. 3.5 is an illustration of these three levels.

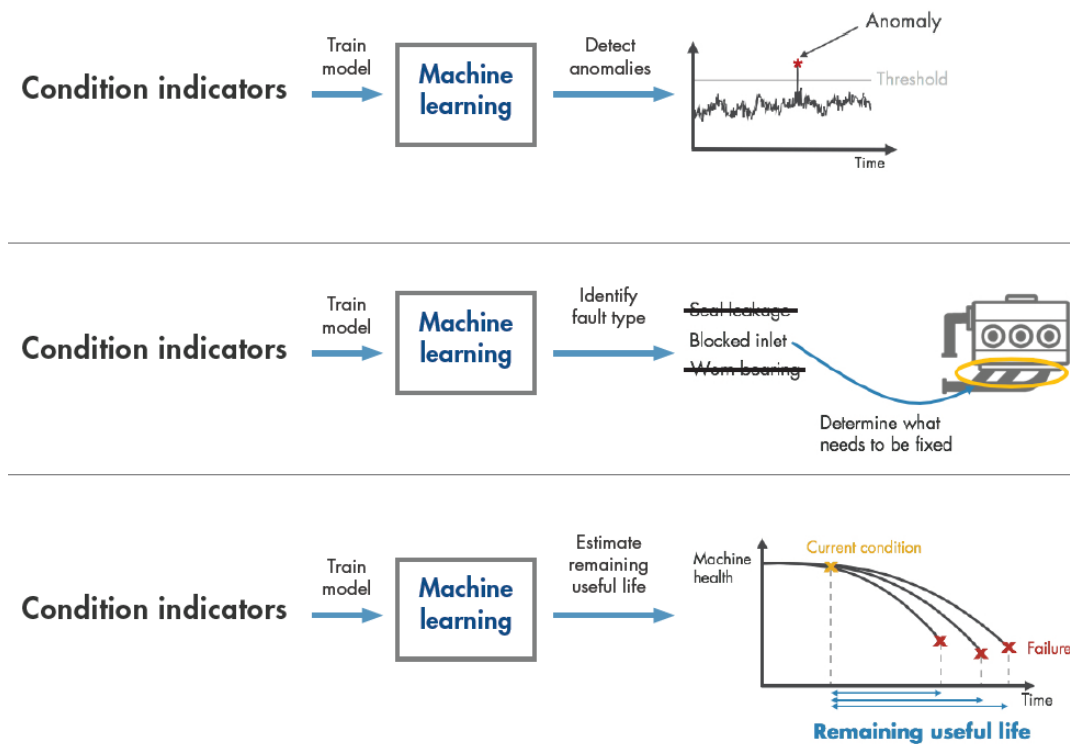


FIG. 3.5: Anomaly Detection, Diagnosis and Prognosis (source: Math-Works)

In summary, Industrial Maintenance can be classified as Fig. 3.6. The level of prediction increases from left to right.

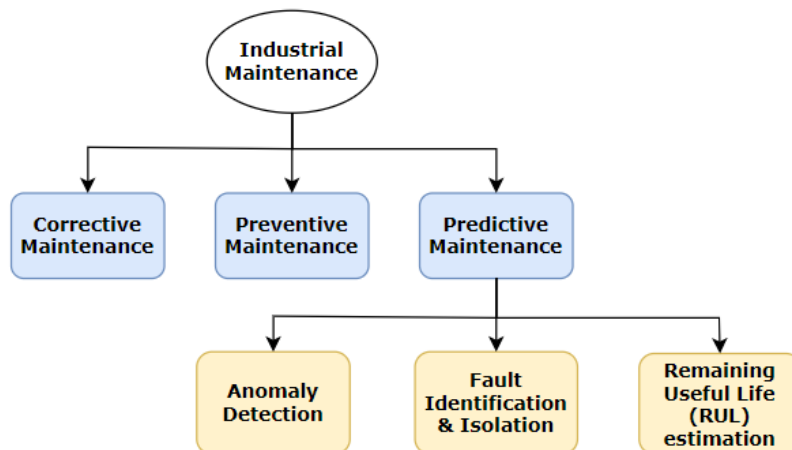


FIG. 3.6: Classification of Industrial Maintenance

3.1.3 Industrial Machine and Machine Data

Because the terminology Industrial Device usually refers to a tangible, stand-alone machine, in this work, the terminology “Industrial Machine” or “Machine” is used for all single industrial devices or group of related industrial devices which conduct a single or a range of industrial tasks. This terminology refers to the wide range of machines used in industrial environment from single equipment to complex machines, from traditional machines without computational capability to smart machines with embedded computer inside, from stand-alone device (e.g. fan, light...) to a group of machine working together as a compound manufacturing unit (e.g. turbine, production line, boiler system...).

In this work, the terminology “Machine Data” refers to all data related to conditions of industrial machines which can be used for Predictive Maintenance. Data can be operational data of machines or information of the machine itself (such as settings, technical specifications). Data can be measured directly or indirectly (e.g. power consumed or product rejection rate). Data can also be information of the environment where machines operate (e.g. temperature, humidity... of the industrial site).

3.1.4 Data-driven approach

Data-driven Predictive Maintenance techniques are based on data of industrial machines: historical data and new operational data. Especially, distinguishing between data of good working condition and data of failure cases is very important.

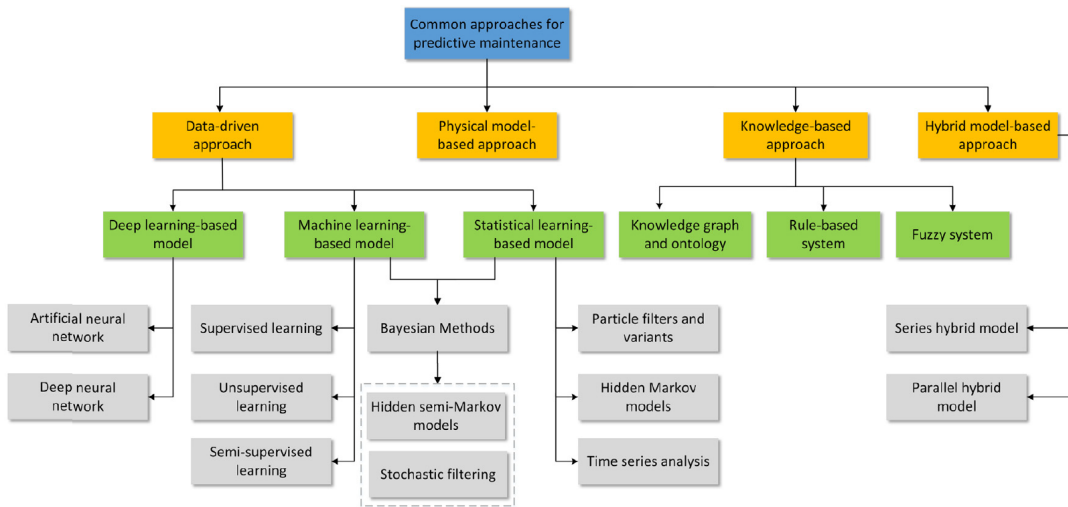


FIG. 3.7: Classification of the common predictive maintenance approaches (Cao et al., 2022)

Basically, Predictive Maintenance methods can be divided into the following three main categories: model-based approach, data-driven approach and knowledge-based approach (Fig. 3.7). The model-based approach uses a mathematical representation of the system combined with the physical understanding of industrial machines. The data-driven approach is preferred when system models are not available due to certain reasons (e.g. machine manufactures do not share or lack of expertise) but machine data is available. Knowledge-based approach implements Predictive

Maintenance solution as Knowledge-based system using techniques such as knowledge graphs, ontology, rule-based or fuzzy logic. There is also the hybrid model-based approach which is a combination of model-based and data-driven approaches an attempts to leverage the strengths from both approaches (Cao et al., 2022).

Although in principle, the model-based approach can provide the most accurate diagnosis and most precise estimation of RUL, it requires a machine knowledge in very high level of detail. So except machine manufactures do it for their own machine, this approach is very difficult to implement. Similarly, knowledge-based approach need the collaboration of machine experts for building the knowledge-based system (for example: define rules).

Therefore, data-driven Predictive Maintenance has attracted wide attention in recent years, especially when industrial wireless sensor networks and industrial cyber physical systems become emerging data acquisition technologies (Susto et al., 2014). The increasing availability of industrial data is boosting the development and deployment of data-driven Predictive Maintenance solutions. It is believed that if we can maximize the usage of these data, we can estimate the health status of an industrial machine or even a complex industrial system, then predict when the maintenance is needed. It is also the context of this work.

In principle, any operational data related to machines can be used for Predictive Maintenance. Hashemian lists several typical type of machine data (but not limited to) which can be used as input for Predictive Maintenance (Hashemian, 2010). Machine data used for Predictive Maintenance can be called Machine Condition Indicators (Fig. 3.8).

Equipment	Vibration	Humidity	Ambient Temperature	Ambient Pressure	Acoustic Signal	Thermography	Motor Current	Insulation Resistance	Electrical Capacitance	Electrical Inductance
Pump	✓		✓	✓	✓	✓	✓	✓		
Valve		✓		✓	✓					
Motor/Fan	✓		✓		✓	✓	✓	✓		✓
Heat Exchangers	✓	✓	✓	✓						
Steam Turbine	✓	✓	✓	✓	✓					
Electrical & Electronic Equipment			✓			✓		✓	✓	✓
Cables and Connectors			✓			✓		✓	✓	✓
Pump Seal		✓		✓	✓			✓		
Piping/ Structures	✓				✓					
Compressor	✓				✓	✓	✓			

FIG. 3.8: Machine Condition Indicators (Hashemian, 2010)

Data can be collected through sensors or through I/O module of machines by Programmable Logic Controller (PLC) of Supervisory Control and Data Acquisition (SCADA) systems or Industrial Distributed Control System (DCS), or machine data can be transmitted directly to data cloud in case of smart machines or smart sensors.

Besides, images (by camera, drone, satellite...) can also be used thanks to machine vision achievements, for example: photos of solar panels in solar power farm or photos of railway.

3.1.5 Examples of Predictive Maintenance application

Following are some studies of Predictive Maintenance. Only purposes of the studies are introduced. Technical aspect and performance are not presented.

Photovoltaic panels (solar panels): Huuhtanen et al. apply Convolutional Neural Network (CNN) to build a regression model for detect anomalies for Photovoltaic panels of solar power plant (Huuhtanen and Jung, 2018).



FIG. 3.9: Photovoltaic panels (source: EuroScientist. Illustration, not used in the study)

Their regression models use historical data of the panel, local weather information and electricity power generated by neighboring panels in order to predict the electricity power that a panel should generate. If the predicted value is different from the real value, usually a drop of amount, there can be a probability that the panel is malfunctioning. In this case, the predictive maintenance problem is modeled as a binary classification model: “functioning” or “malfunctioning”.

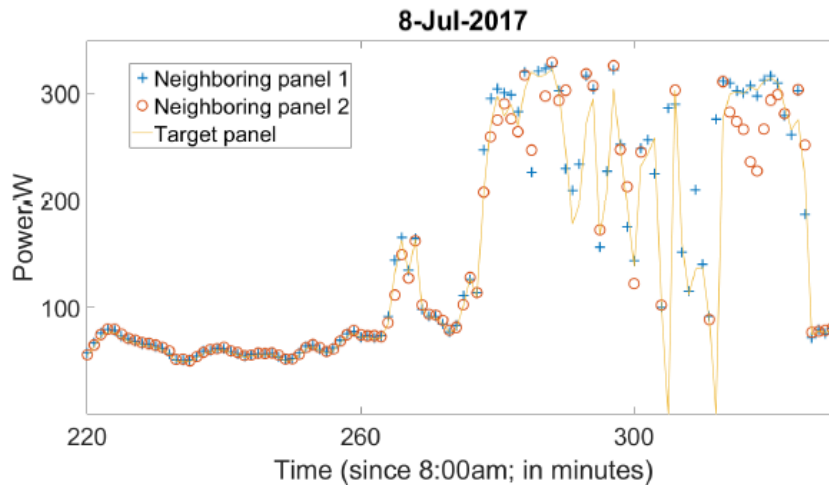


FIG. 3.10: Regression of solar power curves (Huuhtanen and Jung, 2018)

Tungsten filament in semiconductor industry: Susto et al. apply their multiple classifier approach for an industry which really needs predictive maintenance: semiconductor industry (Susto et al., 2014). They tested their methodology for replacing tungsten filaments used in ion implantation, one of the most important processes in semiconductor manufacturing fabrication. In Fig. 3.11, the filament is part of the ion source section of the tool.

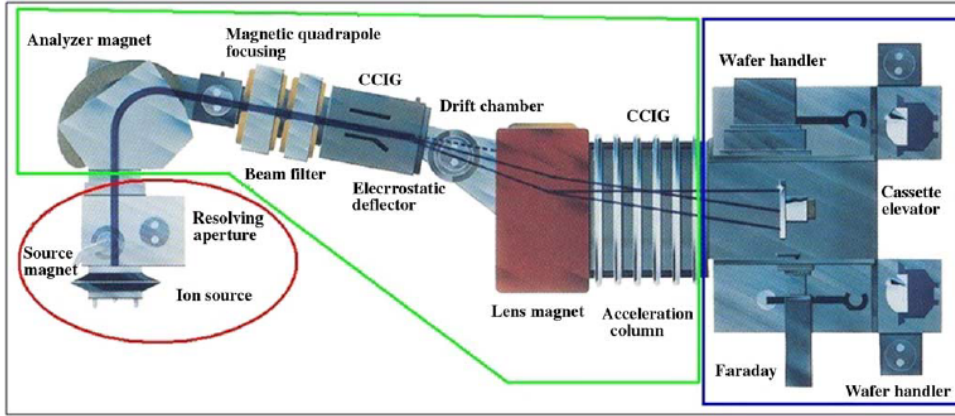


FIG. 3.11: Schematic of a generic ion implanter tool (Susto et al., 2014)

The tungsten filament is considered as a “bottleneck” in production lines due to the high cost of the tool. It must be frequently replaced. Every time a filament is changed, the tool is down for approximately 3h.

Data used in multivariate time series data of 31 variables as following (Fig. 3.12).

Physical/electrical variable	Quantity
Current	9
Current transfer ratio	1
Deceleration	1
Flow	1
Number of scans	1
Plasma gun emission	1
Position	2
Pressure	3
Quantity of electric charge	2
Scan speed	1
Tilt angle	2
Voltage	7

FIG. 3.12: 31 physical/electrical variables (Susto et al., 2014)

They use historical data of these 31 variables to train their multi-class classification model. Then the trained model is used to predict new data in order to classify the degradation level of this component. Because the existing maintenance policy is Run-to-Failure (R2F), there are enough failure data for training the multiple classification model.

3.2 Deep Learning for the Predictive Maintenance problem

The core of the data-driven Predictive Maintenance system are Predictive models. Historical data of the industrial machines are used to build or to train predictive models. Then new operational data will be used to detect anomalies, identify faults and estimate Remaining Useful Life (RUL) of machines in order to predict when maintenance are needed.

Predictive Maintenance problems can be modeled using any Machine Learning technique (Khan and Yairi, 2018). However, Deep Learning is widely used in more and more researches. In this work, Deep Learning is also adopted. The reason why Deep Learning is preferred is presented below.

3.2.1 Concept of Deep Learning

In data-driven approach, two popular methods are learning-based method and statistical-based method. Learning-based method utilizes Machine Learning techniques while statistical-based method is based on statistical models. Although many machine learning techniques are theoretically based on statistical theory, they are usually classified as two different methods (Cao et al., 2022).

Machine Learning, a branch of AI, refers to techniques which have the ability to acquire their own knowledge, by extracting patterns from raw data (or from experience) without being explicitly programmed. Deep Learning is a branch of Machine Learning based on Neural Network.

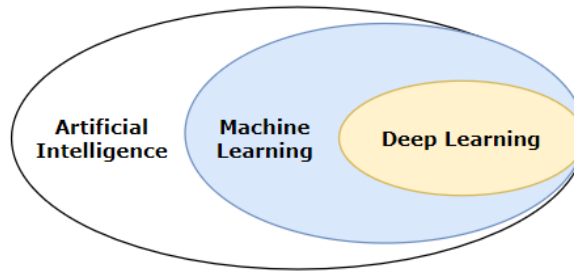


FIG. 3.13: AI, Machine Learning and Deep Learning

For reference, below is the clarification of these concepts by Microsoft (Microsoft, 2022).

Artificial Intelligence (AI) is a technique that enables computers to mimic human intelligence. It includes machine learning.

Machine Learning is a subset of artificial intelligence that uses techniques (such as deep learning) that enable machines to use experience to improve at tasks.

Deep Learning is a subset of machine learning that's based on artificial neural networks that permit a machine to train itself to perform a task. The learning process is deep because the structure of artificial neural networks consists of multiple input, output, and hidden layers. Each layer contains units that transform the input data into information that the next layer can use for a certain predictive task. Thanks to this structure, a machine can learn through its own data processing.

Basically, Deep Learning are machine learning models which are variations of Neural Network. The origin of the name "Deep Learning" is because neural networks used in Deep Learning usually have multiple hidden layers. Fundamental concepts of Deep Learning is presented in 3.2.3.

In terms of implementation, most of traditional Machine Learning are algorithms (e.g. such as Decision Tree, Support Vector Machine, Isolation Forest. . .). Nowadays, the terminology Machine Learning is often used for all Machine Learning techniques which are not Deep Learning.

3.2.2 Supervised, Unsupervised and Semi-Supervised Learning

In terms of learning type, both traditional Machine Learning and Deep Learning can be divided into two main groups: Supervised Learning and Unsupervised Learning (Fig. 3.14). This classification is based on the fact that the dataset used for training the model is labeled or unlabeled.

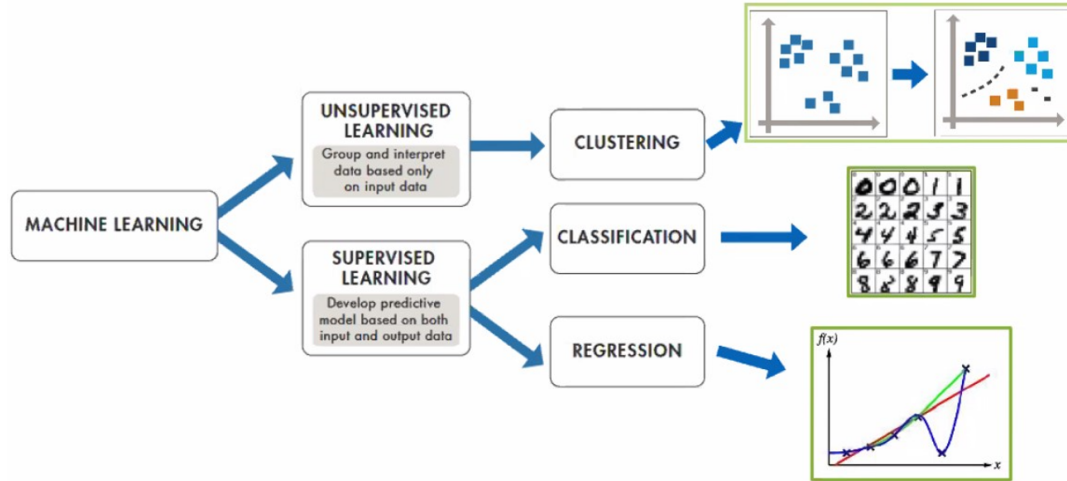


FIG. 3.14: Supervised and Unsupervised Learnings (source: Math-Works)

Classification problem is the task of identifying which group the data belong to. Typical example is image recognition (e.g. flower, fruit, vegetable, animal, vehicle, human face... or object type in general).

Regression problem is the task of prediction of a continuous value (e.g. temperature of climate, fuel consumption of cars, water level of rivers or price of house that clients accept to buy...).

Clustering problem is the task of grouping data based on criteria of similarity. The number of group can be predefined or also discovered by the learning process. Similarly, criteria for grouping can be predefined or can also be discovered during the learning process.

Due to the nature of these problems, classification and regression problems need labelled data while clustering problems can work with unlabeled. In labeled data, the expected result (the label) is known while unlabeled data are just raw data.

Semi-Supervised Learning is the hybrid type between supervised learning and unsupervised learning. This approach combines the majority of unlabeled data with few labeled data. Semi-Supervised Learning has important role in the proposed solution of this work and will be presented in details in 3.3.

Besides, a special branch of Machine Learning is Reinforcement Learning (RL). RL uses the trial-and-error strategy and improves by evaluating the reward (what is the reward and how to evaluate reward depend on the problem). Reinforcement Learning is useful for applications such as Intelligent Agent. Reinforcement Learning can be implemented by Estimated Value Functions or Genetic Algorithms. However, Reinforcement Learning is not considered in this work.

3.2.3 Fundamental concepts of Deep Learning

Perceptron (artificial neuron): Be inspired by the human neural, the simplest unit of neural networks is the Perceptron (Fig. 3.15).

Mathematical formula of a perceptron is:

$$y = f(z) = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

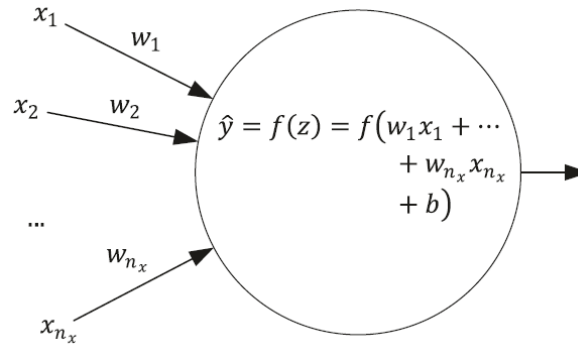


FIG. 3.15: A perceptron (Michelucci, 2018)

Output of an artificial neuron depends on:

- inputs from previous layer $x_1, x_2, \dots, x_n$ (and also recurrent feedback from next layer in case of Recurrent Neural Network)
- weights of each inputs $w_1, w_2, \dots, w_n$
- activation function f . All neurons of a layer have the same activation function. Therefore, activation function is also called Layer Activation Function.
- There is also b which stands for “bias”. Bias is optional because it is actually an adjustment parameter.

Multi-Layer Perceptron (MLP) is the basic Neural Network architecture (Fig. 3.16). It is built by stacking layers of perceptron. MLP consists of at least three layers: an input layer, a hidden layer, and an output layer. MLP is fully connected, because each perceptron receives input from each perceptron in the previous layer and feeds its output to every perceptron in the next layer. Each input will be multiplied with a weight w . All of these weights $w_{i,j}$ are trainable parameters of the neural network.

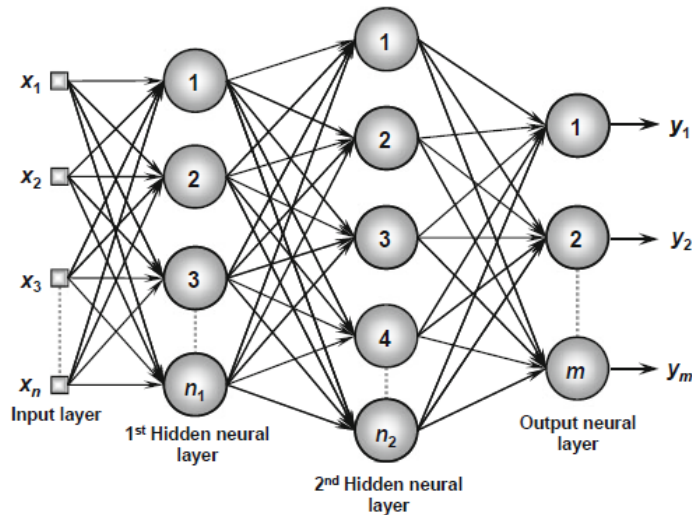


FIG. 3.16: Multi-Layer Perceptron (Da Silva et al., 2017)

MLP is also called Feed Forward Neural Network (FFNN) because output of a perceptron is only sent to next layer and is never be sent back to the previous layer (unlike Recurrent Neural Network architecture, Fig. 3.17). Sometimes the terminology Artificial Neural Network (ANN) is used to refer to FFNN.

Recurrent Neural Network (RNN) is a neural network architecture in which output of a perceptron can be shared back to previous layers (Fig. 3.17). This architecture is needed for processing of sequential data such as language translation, or stock market prediction.

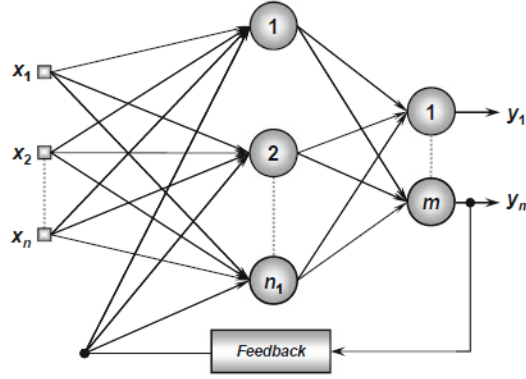


FIG. 3.17: Recurrent Neural Network (Da Silva et al., 2017)

Convolution Neural Network (CNN) is a variation of Feed Forward Neural Network which is very strong in feature extraction. It has two special operations, convolution operation and pooling operation, for extracting features. Although it is originally created for image classification, now it has applications in many other fields including Predictive Maintenance. It will be used in implementation of Deep Learning architectures that need strong feature extraction ability, in case the standard FFNN is not enough powerful.

Generative Adversarial Network (GAN) are Deep Learning architectures that can generate new content based on existing content. A famous application of GAN is DeepFake.

AutoEncoder (AE) is Feed Forward Neural Network type which can reconstruct the input data. AutoEncoder has an important role in proposed solution of this work. Details of AutoEncoder will be presented in this chapter.

Activation function of each neuron is the function f to convert the real weighted input vector to a real value:

$$y = f(z) = f\left(\sum_{i=1}^n w_i x_i\right)$$

Which activation function is selected depends on the problem that needs to be modeled. For example, with unit step activation function, the Neural Network will become a one-class classifier.

Unit step activation function is the simplest activation function (Fig. 3.18). It is also called binary step, threshold or ladder activation function. The threshold can be different from 0 if the bias is added to the neural network.

Identity (Linear) activation function is also a basic activation function with the formula $g(u) = u$ (Fig. 3.19).

Below are popular activation functions used in most of Deep Learning models.

Rectified Linear Unit (ReLU) activation function has output value from 0 to 1 with the following formula $f(z) = \max(0, z)$. **Leaky ReLU** activation function is a variation of ReLU of which the output value can be smaller than 0.

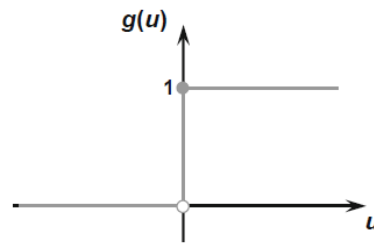


FIG. 3.18: Unit step activation function (Da Silva et al., 2017)

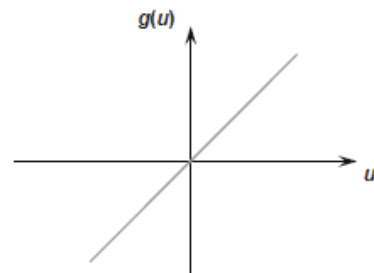


FIG. 3.19: Linear activation function (Da Silva et al., 2017)

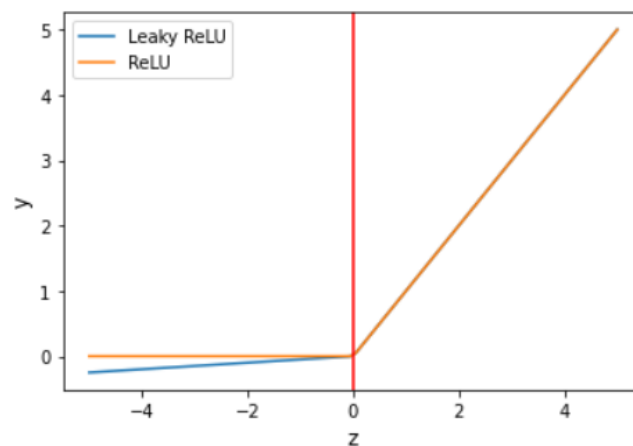


FIG. 3.20: ReLU and Leaky ReLU activation functions

Sigmoid activation function has output value from 0 to 1. **Tanh (Hyperbolic Tangent Activation)** activation function has output value from -1 to 1. They are nonlinear activation functions (Fig. 3.21).

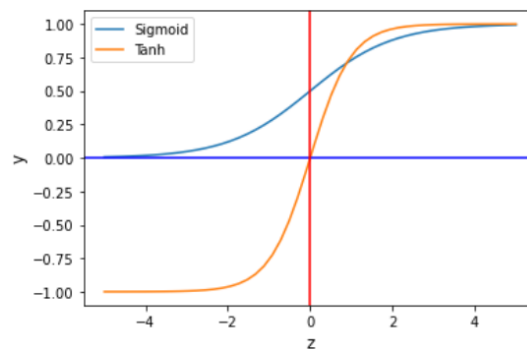


FIG. 3.21: Sigmoid and Tanh activation functions

Besides, there are also specific activation functions which can be used for special purposes by advanced experts, for example: Gaussian activation function (Fig. 3.22).

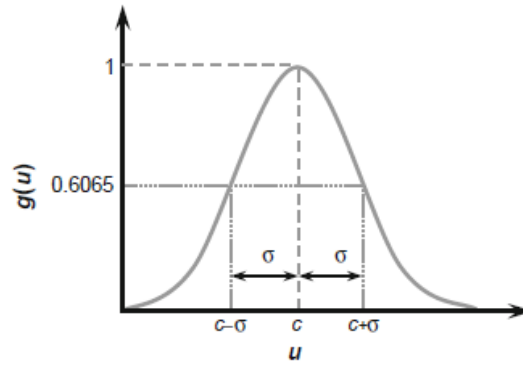


FIG. 3.22: Gaussian activation function

Softmax activation function for classification: Softmax activation function is a special activation function used for the last layer of classification models.

The Classification model has to capture an underlying function $y = f(x)$ from the training dataset in which:

$x = (x_1, x_2, \dots, x_n)$ is input data. Input can be a multivariate data sample or a data frame (a sequence of data sample).

$y = (y_1, y_2, \dots, y_m)$ is output. They are probabilities that the input belongs to each classified class.

So sum of outputs of the last layer must be 100%.

$$y_1 + y_2 + y_3 + \dots + y_m = 1$$

Therefore, regardless the implementation of input layer and hidden layers, the last of the Classification model often:

- is fully connected with the previous layer. Each neuron receives inputs from all neurons of the previous layer.
- has softmax activation function which converts a vector of values to a probability distribution

Softmax activation function converts a real vector to a vector of categorical probabilities. The elements of the output vector are in range $(0, 1)$ and sum to 1. Softmax is often used as the activation function for the last layer of a classification network because its special characteristic can be used to represent the probability distribution (<https://keras.io/api/layers/activations/>).

The formula of Softmax activation function ensures that total distribution of all probabilities is 100% (Michelucci, 2018).

$$S(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Training the Neural Network is a process of adjusting all weights $w_{i,j}$ of all perceptrons of all layers so that output of the last layer is as close as possible to expected output.

This objective can be done thanks to Backward Propagation algorithms implemented in any Deep Learning programming framework. Most of Backward Propagation algorithms are based on Gradient Descent algorithm, an iterative optimization algorithm.

During the training process, for each sample batch of training data, the computation includes

- Calculate the output
- Calculate the distance between the output and the expected result
- Run a Backward Propagation algorithm to adjust all trainable parameters so that the output will be a bit closer to the expected result

These steps will be repeated until the whole training dataset is processed. One time that the whole dataset is processed is called an **epoch**. The number of epoch of the training process can be hundred or even thousand.

The number of epoch increases until the accuracy cannot be improved. It means the training process is converged and this state is called the convergence.

Training Neural Network is a very heavy process. Let's examine a simple Deep Learning model for classification of animal images. This is a standard FFNN with input as 200x200 pixel images (reshaped as one-dimension array of 40,000 real numbers) and output are 100 animal types. It has 5 hidden layers, each layer has 64 perceptrons. This model has up to 2,5 million trainable parameters (Fig. 3.23).

```
inputs = keras.Input(shape=(40000,), name="digits")
x = layers.Dense(64, activation="relu", name="dense_1")(inputs)
x = layers.Dense(64, activation="relu", name="dense_2")(x)
x = layers.Dense(64, activation="relu", name="dense_3")(x)
x = layers.Dense(64, activation="relu", name="dense_4")(x)
x = layers.Dense(64, activation="relu", name="dense_5")(x)
outputs = layers.Dense(100, activation="softmax", name="predictions")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
model.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #
digits (InputLayer)	[(None, 40000)]	0
dense_1 (Dense)	(None, 64)	2560064
dense_2 (Dense)	(None, 64)	4160
dense_3 (Dense)	(None, 64)	4160
dense_4 (Dense)	(None, 64)	4160
dense_5 (Dense)	(None, 64)	4160
predictions (Dense)	(None, 100)	6500
Total params: 2,583,204		
Trainable params: 2,583,204		
Non-trainable params: 0		

FIG. 3.23: A Feed Forward Neural Network structure and parameters

If the training dataset contains 10,000 images and the model needs to be trained 100 (can be more) cycles (epochs) before being converged, it takes much time and computational resource. Moreover, this is standard Neural Network without recursive mechanism. With more complicated models such as Long Short-Term Memory (LSTM), the computation is much heavier.

Training the Neural Network is all called the **fitting** process, because it makes the model fitted in the training dataset. Once a model is fitted with the training dataset, this model can predict well all data sample of this dataset. Usually the accuracy need to be at least 90% so that it can be deployed to production.

Hyperparameter: During the training process, many other parameters need to be set such as number of epoch, batch size, optimization algorithm, loss function... These parameters don't belong to the neural network but are used to control the training process. They are called hyperparameter. The process of selecting the best combination of hyperparameters for training is called hyperparameter turning.

Prediction: Once the model is trained, it can be used for prediction. Prediction of a data sample is to pass this data sample as input of first layer of the model and to receive the output from the last layer of the model. Depending on the model, the prediction output will be a single value or an array of value. Prediction can be done for a single data input or a whole input dataset at the same time. Prediction result of a whole dataset, which is an array of data sample, will be simply an array of prediction output.

Although training the neural network is heavy, using the trained model for prediction is quite fast. Prediction process is as fast as calling a numerical function with a chain of simple numerical and logical operations thanks to the structure of the MLP. All computation is calculated in memory and inside of the Neural Network. CNN and LSTM are more complicated, but still fast enough for systems that need to provide quick response.

Generalization: A good generalization is the objective when building a Deep Learning model. A Deep Learning model generalizes well a problem (or a real system) if it not only predicts correctly the training dataset but also predicts well new data.

Underfitting and Overfitting: If the model cannot generalize well the problem (or the real system), it can fall into underfitting or overfitting case. Underfitting is the case that the training process cannot be converged with the training dataset. Overfitting is the case that the model predicts too perfectly with the training dataset but works badly with new data. There are many specific ways and techniques to overcome these two challenges.

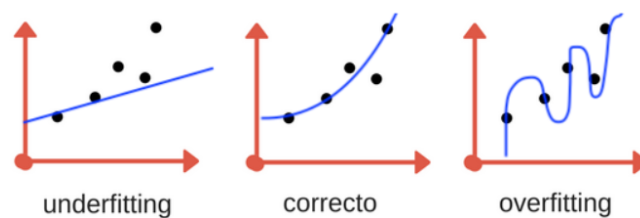


FIG. 3.24: Underfitting versus Overfitting (source: aprendemachine-learning.com)

3.2.4 Why Deep Learning is preferred for Predictive Maintenance?

The core of the data-driven Predictive Maintenance system are Prediction models which are trained by historical data of industrial machines and then can predict necessary maintenance. In principle, any Machine Learning technique can be used to implement these predictive models. However, Deep Learning is preferred research in recent years.

Compared to traditional Machine Learning algorithms, Deep Learning models require more data, computational resource and time to train but requires less domain knowledge (Fig. 3.25).

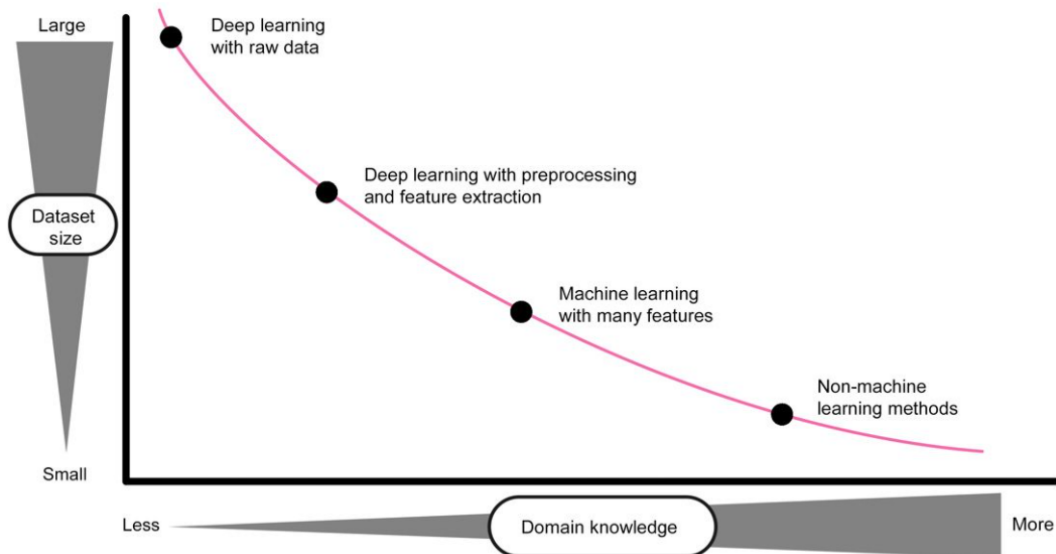


FIG. 3.25: Deep Learning versus Machine Learning
(<https://es.mathworks.com/campaigns/offers/next/practical-guide-to-deep-learning/collecting-data.html>)

The performance of Deep Learning relies heavily on the training data. However, with the development of IIoT, big industrial dataset for Deep Learning is no longer rare, although there are still challenges with data quality and data fusion.

Zhang et al. compare the flows of applying Traditional Machine Learning and Deep Learning for Predictive Maintenance and explain why Deep Learning is more adopted (Zhang, Yang, and Wang, 2019). The reason is Deep Learning can avoid complex feature engineering steps that need domain knowledge of machine and process. In Fig. 3.26, we can see that traditional Machine Learning is a bit like the hybrid approach between model-based approach and data-driven approach. It is more suitable for specific, critical and expensive industrial systems than for building a general and scalable Predictive Maintenance solution. Instead, Deep Learning moves directly to an end-to-end learning.

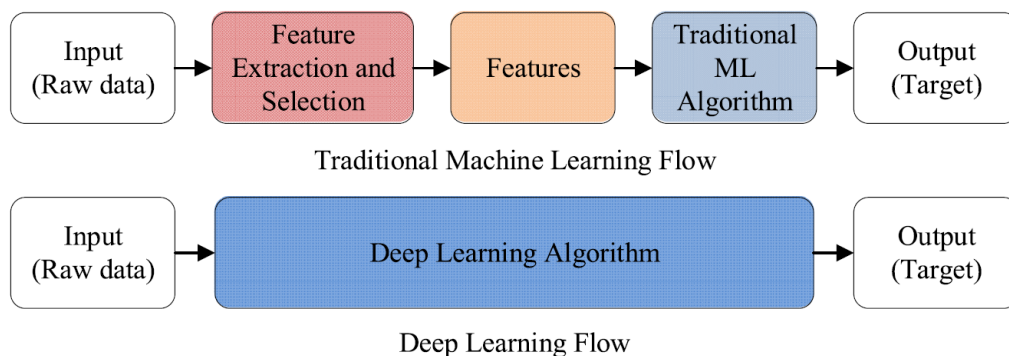


FIG. 3.26: Machine Learning and Deep Learning flows

Olga Fink also analyzes reasons why Predictive Maintenance solutions need to

change from feature engineering to feature learning due to the large number of different types of sensors, resulting a very heterogeneous condition monitoring data. Traditional machine learning which requires manual feature engineering steps show many limitations such as limited scalability or dependence on domain experts. End-to-end learning architectures of deep learning allows to use directly raw data input, with automatic feature learning steps within the neural network architectures (Fink, 2020).

Deep Learning is more suitable for building a general and scalable Predictive Maintenance solution which is the ambition of the research community. This is the reason why Deep Learning is chosen in this work.

Conclusion: In order to model a problem or a physical system using Deep Learning for a certain purpose, including Predictive Maintenance, it is necessary to:

- Choose or create the suitable Deep Learning model structure
- Have to good training dataset
- Build an effective training process

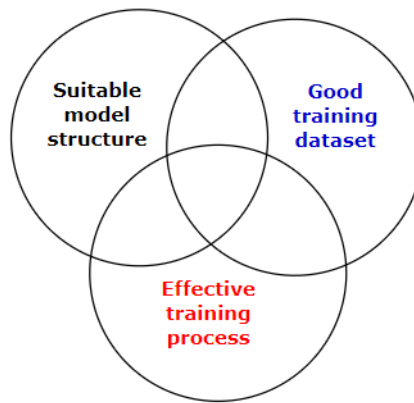


FIG. 3.27: Conditions for of Deep Learning approach

Selection of the suitable model structure means to decide which is the best way to model the problem (classification, regression, clustering, generative, autoencoder...) then decide which basic Deep Learning model (FFNN, CNN, LSTM...) should be used to implement the designed model. After that, how many layers, size of each layer, activation function for each layer... also need to be optimized.

A good training dataset is not only big enough but also represents well the problem (or the real physical system). Requirements for a good training dataset depends on the selected model and vice versa, selection of model must refer to available training data.

Finally, effective training process is crucial for having a useful model, including not only hyperparameter turning but also data preprocessing. Deep Learning models not only needs to be trained one time but also need be re-trained periodically. Otherwise, the model will be drifted (obsolete) due to lack of training with recent data.

Although Deep Learning has an important role in the proposed solution of this work, the primary topic of this TMF is not Machine Learning nor Data Science. Therefore, in this work, we will not try to explain, improve or invent new Deep

Learning models. Instead, we utilise Deep Learning models justified by state-of-the-art studies and follow best practices recommended by research community and professional experts.

3.3 Semi-Supervised Learning for Anomaly Detection

3.3.1 Supervised learning methods for Predictive Maintenance

Although supervised learning approach is not chosen in this work, some studies of Predictive Maintenance using supervised learning method are briefly presented, in order to understand why supervised learning is not always applicable in real life.

In the predictive maintenance flow proposed by Rivas et al., there are two steps (Rivas et al., 2019). First, a binary classification model classifies if the operating cycle before failure of the machine is bigger or smaller than 100. If the number of machine cycle before failure is smaller than 100, in the second step, a regression model will estimate RUL by predicting the concrete number of cycle. Both classification and regression models are implemented by LSTM (Fig. 3.28). Dataset C-MAPSS, a Run-to-Failure experimental dataset, is used to testing (Saxena and Goebel, 2008). In the C-MAPSS dataset, the label is the number of cycle before the engine crashes.

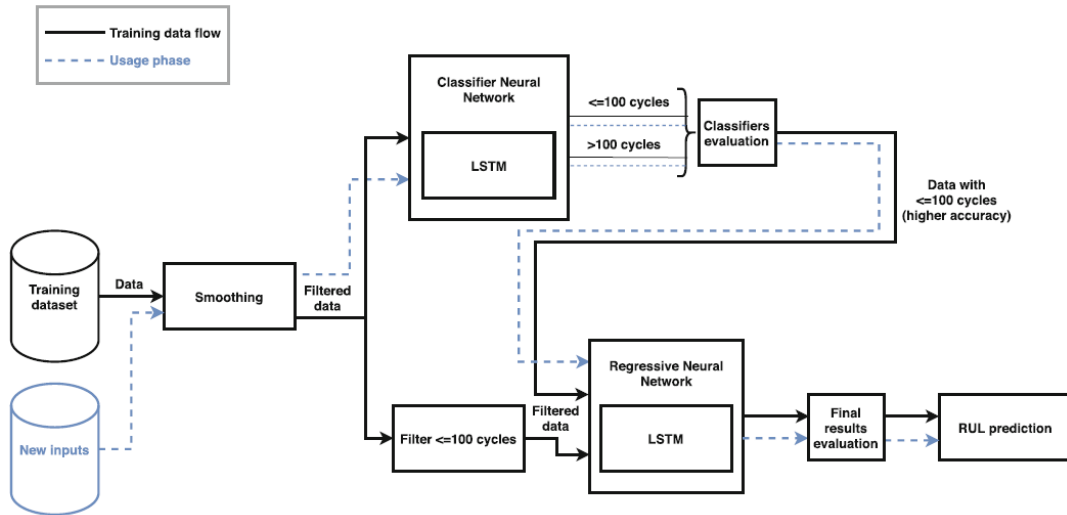


FIG. 3.28: Classification and Regression models implemented by LSTM (Rivas et al., 2019)

Meesublak et al. uses one-dimension Convolutional Neural Network (1D CNN) to classify conditions of bearing (Meesublak and Klinsukont, 2020). Bearing is the key components of rotating machinery (Fig. 3.29).

1-D CNN is a simplified version of the standard two-dimension CNN (2-D CNN) created for image classification (Fig. 3.30). Input is 1-dimension array of univariate vibration signal.

Dataset used for testing is the bearing vibration data collected by the Bearing Data Center at Case Western Reserve University. The vibration signal data were monitored in at the frequency of 12,000 samples per second. Output of the model are labels as Fig. 3.31.

Six labels are categorical conditions of bearing: Normal Baseline (NB), Inner Race Fault (IF), Ball Fault (BF), Outer Race Centered Fault (CF), Outer Race Orthogonal



FIG. 3.29: Ball bearing (source: Fractory. Illustration, not used in the study)

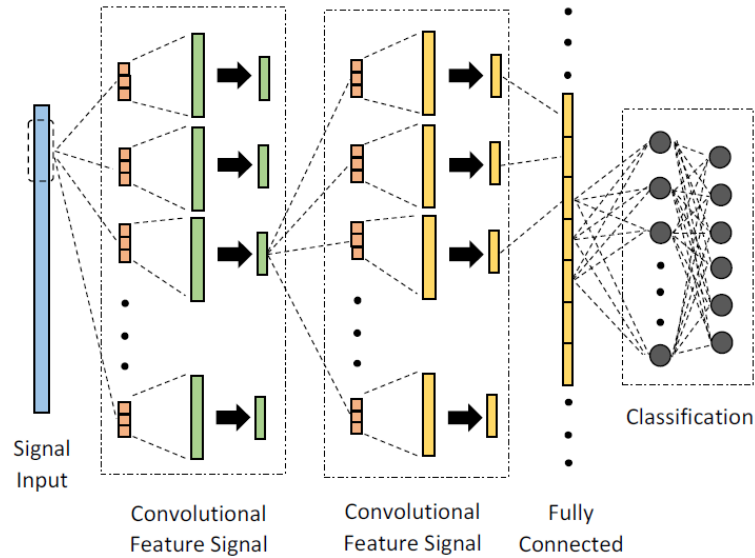


FIG. 3.30: Classification model implemented by CNN (Meesublak and Klinsukont, 2020)

		Predicted Label					
		NB	IF	BF	CF	ORF	OPF
True Label	NB	200	0	0	0	0	0
	IF	0	192	0	0	0	8
	BF	0	0	200	0	0	0
	CF	0	0	0	129	71	0
	ORF	0	0	0	96	101	3
	OPF	0	0	0	0	0	200

FIG. 3.31: Bearing condition classification (Meesublak and Klinsukont, 2020)

Fault (ORF), and Outer Race Opposite Fault (OPF). Because bearing is quite standardized component (it has the same design principle in any machine), it is expected that the trained model can be used for real cases.

3.3.2 The need of Semi-Supervised learning

Although Classification and Regression models can produce very positive results, we can see that very few supervised learning studies can be tested with real life case studies because these models need to be trained by labelled dataset. Especially, the dataset also needs to be balanced with 20%-30% (or at least 5%-10%) data of failures.

The real balanced dataset like that is very rare in industrial environment because very few (or even zero) failures is the objective of industrial management.

Therefore, most of studies using Supervised Learning approach have to use experimental datasets to verify their Deep Learning models. For example, Meesublak et al. use vibration signal of bearing in database collected by the Bearing Data Center at Case Western Reserve University (Meesublak and Klinsukont, 2020). Especially, many studies develop their models using the C-MAPSS dataset which is a Run-to-Failure experimental dataset shared by NASA (Saxena and Goebel, 2008). To create this dataset, NASA runs aircraft engines from new condition until they crash. During this progress, vibration signals are measured and the number of cycle before failures is used as data label.

We never have much failure data in real world because machine failure is rare event in nature. Sometimes it does not exist, especially with new industrial systems, and when it happens, label is often inconsistent or ambiguous (Serradilla et al., 2021). For example, below is real failure information of a wind power plant during the year 2016.

Wind Turbine failures in 2016			
Turbine_ID	Component	Timestamp	Remarks
T01	GEARBOX	2016-07-18T02:10:00+00:00	Gearbox pump damaged
T06	GENERATOR	2016-07-11T19:48:00+00:00	Generator replaced
T06	GENERATOR	2016-07-24T17:01:00+00:00	Generator temperature sensor failure
T06	GENERATOR	2016-09-04T08:08:00+00:00	High temperature generator error
T06	GENERATOR	2016-10-27T16:26:00+00:00	Generator replaced
T06	GENERATOR	2016-10-02T17:08:00+00:00	Refrigeration system and temperature sensors in generator replaced
T06	HYDRAULIC_GROUP	2016-04-04T18:53:00+00:00	Error in pitch regulation
T07	GENERATOR_BEARING	2016-04-30T12:40:00+00:00	High temperature in generator bearing (replaced sensor)
T07	TRANSFORMER	2016-07-10T03:46:00+00:00	High temperature transformer
T07	TRANSFORMER	2016-08-23T02:21:00+00:00	High temperature transformer. Transformer refrigeration repaired
T09	GEARBOX	2016-10-11T08:06:00+00:00	Gearbox repaired
T09	GENERATOR_BEARING	2016-06-07T16:59:00+00:00	High temperature generator bearing
T09	GENERATOR_BEARING	2016-08-22T18:25:00+00:00	High temperature generator bearing
T09	GENERATOR_BEARING	2016-10-17T09:19:00+00:00	Generator bearings replaced
T11	GENERATOR	2016-03-03T19:00:00+00:00	Electric circuit error in generator
T11	HYDRAULIC_GROUP	2016-10-17T17:44:00+00:00	Hydraulic group error in the brake circuit

FIG. 3.32: Wind Turbine failure information (Inovação, 2018)

We can see that failures are not only rare but also poor described without error classification, severity level, duration of downtime, or clear border between good and failure statuses...

By all of above reasons, studies on semi-supervised learning approach for the Predictive Maintenance problem are increasing in recent years. Semi-supervised learning is an machine learning approach that needs only a small amount of labeled data combining with a large amount of unlabeled data for training the models. Labeled data can be real or simulated data for verifying the trained models.

3.3.3 Semi-supervised Anomaly Detection

Most of solutions for Fault Classification (Diagnosis) and RUL (Prognosis) problems are supervised learning models which need labelled and balanced data for training. Unsupervised learning methods for these problems is very rare.

However, for the first level of Predictive Maintenance which is Anomaly Detection (or Outlier Detection) (Fig. 3.34), the unsupervised learning approach is more popular. Anomaly detected can be point anomaly or pattern anomaly. Anomaly

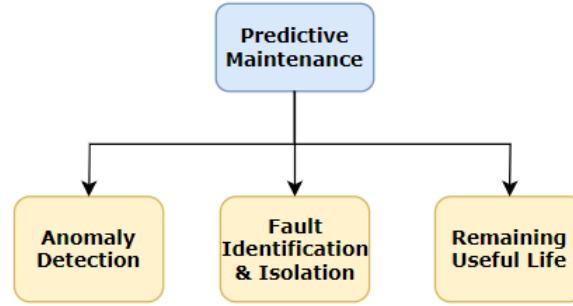


FIG. 3.33: Three levels of Predictive Maintenance

detection methods can vary from simple thresholds of single signal to statistics, machine learning or deep learning models. For example, Zhao et al. use Correlation-Based method for anomaly detection (Zhao et al., 2017a). They calculate correlations between signals of machines in good conditions and continue to monitor these correlations. Compared to the simple approach of signal threshold, the correlation is more sensitive to anomalies because it not only monitors if sensor signals are out of normal range but also monitors if the correlations are out of normal range. It can realize anomalies even if all signals are still in normal range.

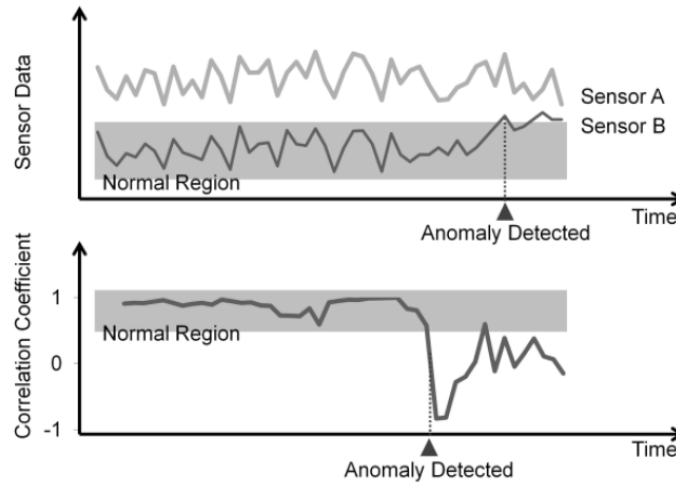


FIG. 3.34: Anomaly detection using correlation coefficient (Zhao et al., 2017a)

In general, an anomaly in a system is any abnormal behavior of the system. According to Lopez et al., *an anomaly is an occurrence that is different from what is standard, normal, or expected* (Lopez et al., 2017). As shown in Fig. 3.35, the root cause of anomalies is not only machine failures. The reason can also be broken sensors, errors of SCADA system/network, cyber-attacks and so on.

According to Lopez et al., Semi-supervised anomaly detection are Semi-supervised methods use normal data sets to train models of normal behavior, and label any observations that deviate significantly from those models as anomalies (Lopez et al., 2017).

In a data-driven anomaly detection system, a high occurrence of anomalous data can be considered as anomalies. For Predictive Maintenance problem, anomalies can be early signs of machine failures.

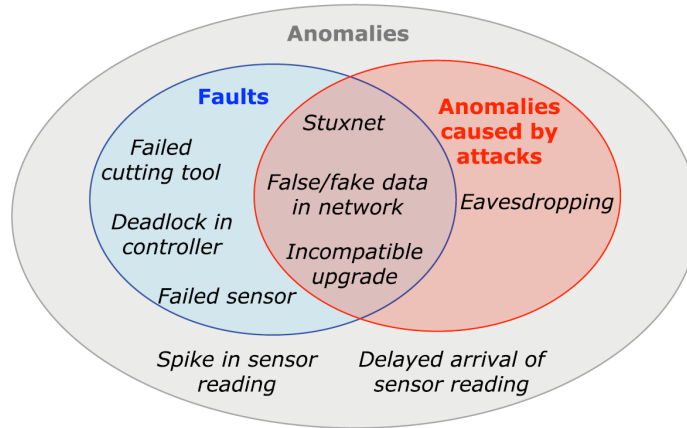


FIG. 3.35: Anomalies in Smart Manufacturing (Lopez et al., 2017)

In principle of semi-supervised Anomaly Detection methods, the model will be trained with data of normal conditions in a certain way so that it can detect if new data is anomalous. In some researches, Anomaly Detection model is also called Normal Behavior Model (NBM) because it learns machines or systems in normal conditions (Roelofs et al., 2021) (Serradilla et al., 2021).

Fig. 3.36 is a general learning and prediction process of a Normal Behavior Model (NBM) presented by Pollak et al. (Pollak et al., 2021).

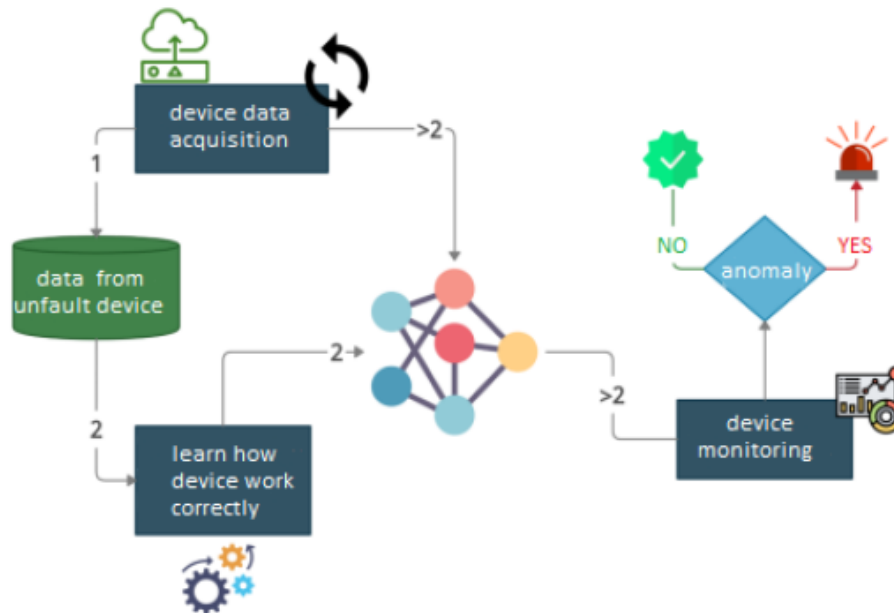


FIG. 3.36: Normal Behavior Model for Anomaly Detection (Pollak et al., 2021)

Critical condition of this approach is that the normal behavior model must be trained by normal data. Normal data is data of un-fault device (or data of normal working conditions) (Pollak et al., 2021) (Tedesco et al., 2021). If there are anomalous data mixed in training data, the model will receive undesired training and not be able to recognize anomalies effectively in the future.

Machine data is acquired by Wireless Sensor Network (WSN) and SCADA system. WSN and SCADA are parts of the industrial system. They can also have failures

and can generate anomalous data. There are studies about anomaly detection for SCADA system (Nazir, Patel, and Patel, 2021) and WSN (Luo and Nagarajan, 2018). However, it is not in the scope of this work. Most of studies of anomaly detection for industrial machines rely on the condition that WSN and SCADA work well. In case there are anomalous data generated by other reason, anomaly notification raised by the model is simply skipped because they are false alarms. If these anomalous data were used for training the model, they must be excluded from training data and the model must be re-trained.

How to select normal data for training and re-training the models will be presented in details in Chapter ?? . It is an important aspect of using predictive models in real life.

Various techniques of different theories can be used to model the anomaly detection problem. For example, Zhao et al. use statistical model to discover statistical distributions of correlations of sensor data (Zhao et al., 2017a). Among Machine Learning algorithms, the current best-in-class Outlier Detection algorithm is Isolation Forest (Alfeo et al., 2020).

Two popular learning-based methods are Regression model and AutoEncoder. In Regression method, the models try to predict an output or a dependent variable using related input variables, while AutoEncoder-based models try to reconstruct input data. Regression models can be implemented by either Machine Learning algorithms or Deep Learning models, while AutoEncoder is Deep Learning model. Although Regression method is not chosen in this work, it is briefly presented here for comparison.

3.3.4 Regression models for Anomaly Detection

Moleda et al. build an anomaly detection system for early detection of faults in Boiler Feed Water Pumps in power plant (Moleda, Momot, and Mrozek, 2020). These pumps are parts of continuous manufacturing process of electricity production (Fig. 3.37).

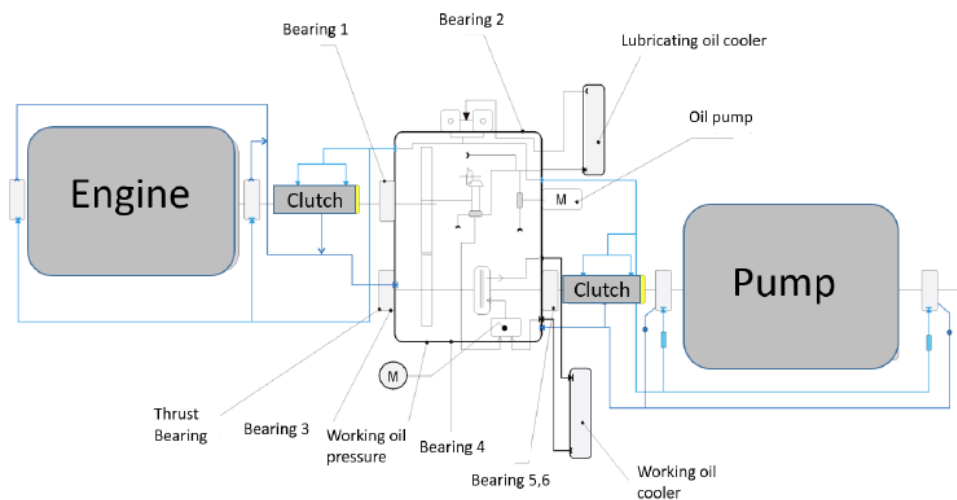


FIG. 3.37: Boiler Feed Water Pumps in power plant (Moleda, Momot, and Mrozek, 2020)

Moleda et al. implement their regression model using their own algorithms. Input are 22 variables, mostly temperature and pressure, while output is value of

the water flow behind the pump. The regression model is trained with historical data of good conditions acquired by SCADA system.

In order to predict anomalies, predicted values by the regression model are compared to real values. If the gap between predicted values and real values of water flow has the trend to increase, it can be an early sign of failures.

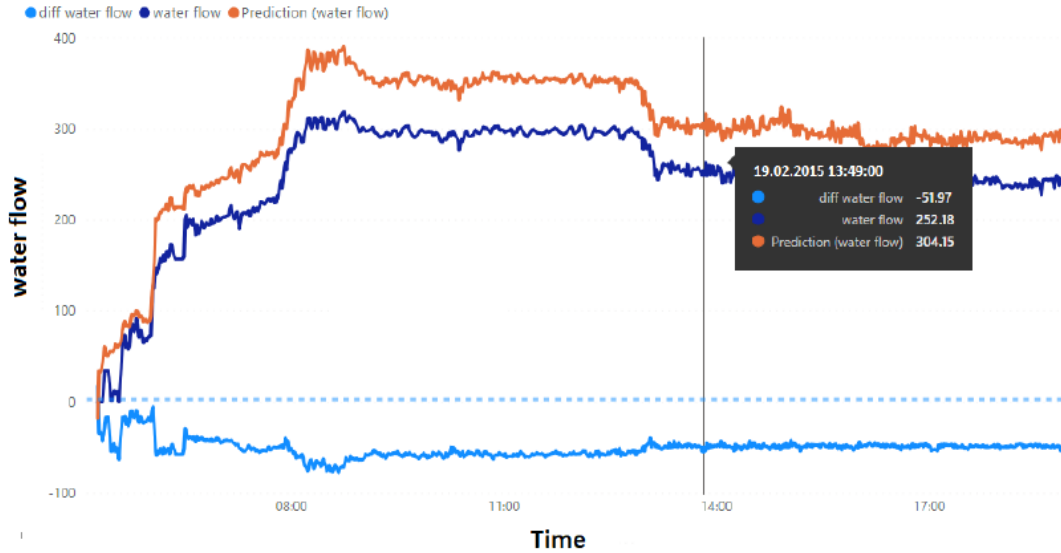


FIG. 3.38: Real and estimated value of the water flow behind the pump (Moleda, Momot, and Mrozek, 2020)

Another example of regression method is regression model for detect anomalies for photovoltaic panels of solar power plant by Huuhtanen et al. (Huuhtanen and Jung, 2018). In this case, output is amount of electric power a panel should generate while input is historical data of that panel, local weather information and electricity power generated by neighboring panels. The regression model is implemented by Convolutional Neural Network (CNN). However, regardless which technique is used, the idea is similar.

Regression method, in case it can be developed, will be very efficient. It usually uses selective input variables for training the regression model, leading to computational simplicity. Regression model can be implemented by both Machine Learning or Deep Learning. Especially, its result is very human readable, without sophisticated interpretation.

However, in regression method, an explicit output, or a dependent variable, needs to be known. This condition can be difficult, or it's even impossible, in many cases (e.g. belt drive in study by Pollak et al. (Pollak et al., 2021)).

Moreover, in order to select input and output variables and verify their dependency, a certain level of knowledge of the machine is needed (Fig. 3.37, for example). This is a little like the hybrid approach between physical model-based approach and data-driven approach.

For building a data-driven anomaly detection solution with minimal requirement of machine knowledge, the AutoEncoder-based method is more suitable. The approach using AutoEncoder is more general and scalable. As a trade-off, it will be heavier because AutoEncoder is a complex Deep Learning architecture.

For these reasons, AutoEncoder-based method is chosen for the Anomaly Detection solution in this work.

3.4 AutoEncoder for Anomaly Detection

3.4.1 Anomaly Detection based on Data Reconstruction Error

As illustrated in Fig. (Fig. 3.39), in semi-supervised anomaly detection approach, after models are trained with machine data of normal operating conditions, machine health indicators of normal data will be calculated and then a threshold will be set. With new data, these indicators will be calculated. If values of these indicators of new data are above the threshold, new machine data are considered as anomalous data and can be signs of coming failures.

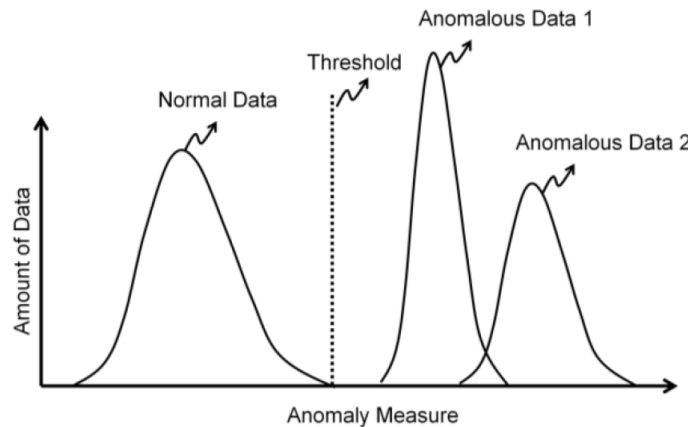


FIG. 3.39: Anomaly Detection (Zhao et al., 2017a)

Percentage of anomalous data (having health indicators above the threshold) is called Anomaly Measure (or Anomaly Score). Depending on the chosen technique, the anomaly score is monitored and if it is higher than the threshold of normal condition during a continuous monitoring time range, the current condition of the machine can be anomaly.

In the Deep Learning approach using AutoEncoder, the anomaly score is calculated from data Reconstruction Error (Fig. 3.40).

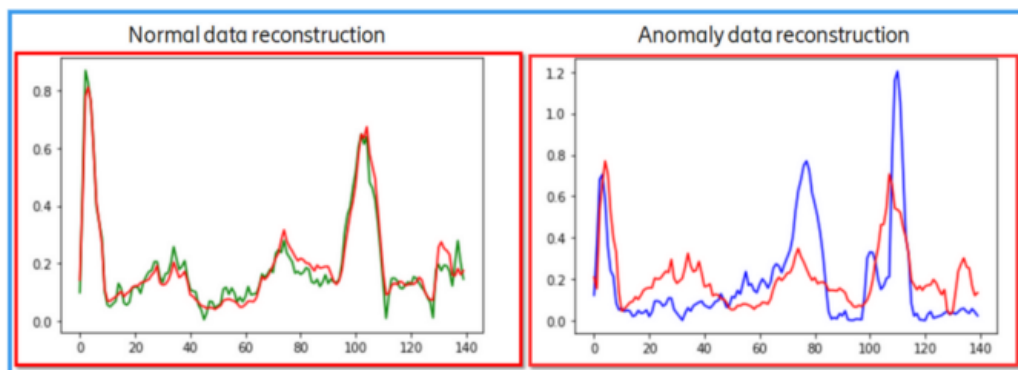


FIG. 3.40: Anomaly Detection based on Data Reconstruction Error (https://medium.com/@Sanath_raj/anomaly-detection-using-autoencoders-ad92aa0493d3)

3.4.2 Introduction of AutoEncoder

AutoEncoder (AE) are Neural Network type that can reconstruct the input. An Autoencoder is a feed forward neural network in which the desired output is the input itself. AE is able to tackle unsupervised tasks, and for this reason are strongly used for anomaly detection (Tedesco et al., 2021). Fig. 3.41 is the basic schematic diagram of the AutoEncoder.

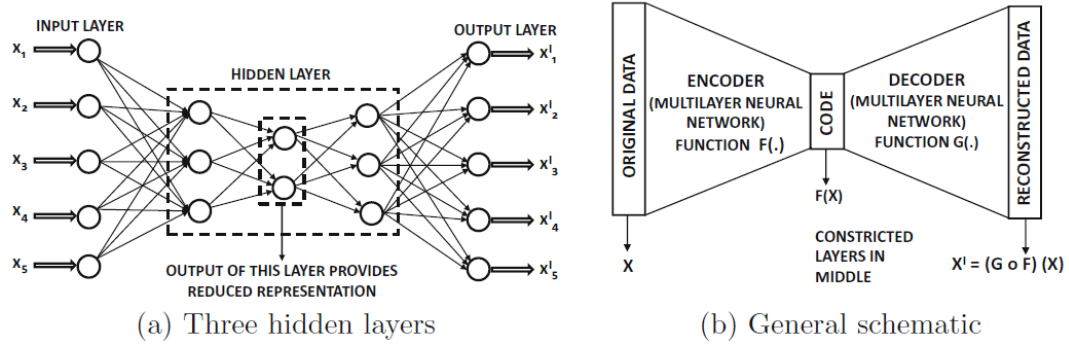


FIG. 3.41: The basic schematic of the autoencoder (Aggarwal et al., 2018)

The typical AE structure is subdivided into three main parts (Tedesco et al., 2021):

- Encoder: neural network that has the goal to learn how to encode the original data in a low dimensional representation;
- Bottleneck Layer: the layer that contains the coded reduced representation. Content of this layer is the latent representation of input data.
- Decoder: neural network that has the goal to reconstruct from the bottleneck representation the original data in the most possible way.

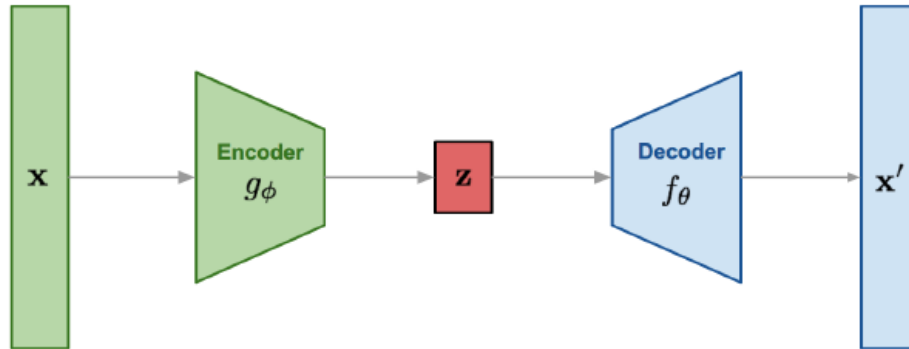


FIG. 3.42: Structure of AutoEncoder (Pollak et al., 2021)

Fig. 3.42 is the mathematic presentation of the AutoEncoder, of which:

- x is original input data.
- z is the **latent representation** (reduced dimensional data) of the original data: $z = g(x)$. z is also called **latent vector**.

- x' is the reconstructed data from reduced dimensional data: $x' = f(z)$

The full data transformation of the AutoEncoder will be: $y = f(z) = f(g(x))$

Because in AutoEncoder, expected output is also input, the objective of the training process will be:

$$x' = x \Leftrightarrow f(g(x)) = x$$

The distance between original input data x and reconstructed data x' is Reconstruction Error (RE). RE is measured by loss function $L(x, x')$. L can be simple distance formula such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) or more complicated distance formula such as Mahalanobis distance (Lutz et al., 2020). The most popular distance used is MSE:

$$L(X, X') = \frac{1}{n} \sum_{i=1}^n (x_i - x'_i)^2$$

Training the AutoEncoder is the progress of minimizing the lost function $L(X, X')$.

In order to achieve this objective, the structure of an AutoEncoder must comply these constraints:

- Input and output layers (first and last layers) must have the same size
- The size of the latent representation layer must be much smaller than the size of the input and output layers (otherwise, it is just the copy of the input)
- The number of nodes in each layer decreases from the input layer to the representation layer and then increases from the representation layer to the output layer

Besides these constraints, the implementation of AutoEncoder is quite free. Therefore, AutoEncoder is actually a Deep Learning architecture more than a concrete Deep Learning type because it can be implemented by any of three popular Deep Learning types: Feed Forward Neural Network (MLP), Convolutional Neural Network (CNN) or LSTM.

There are variations of AutoEncoder. The standard form is UnderComplete AutoEncoder (UC AE). It can be implemented by a basic FFNN ((Fig. 3.43)).

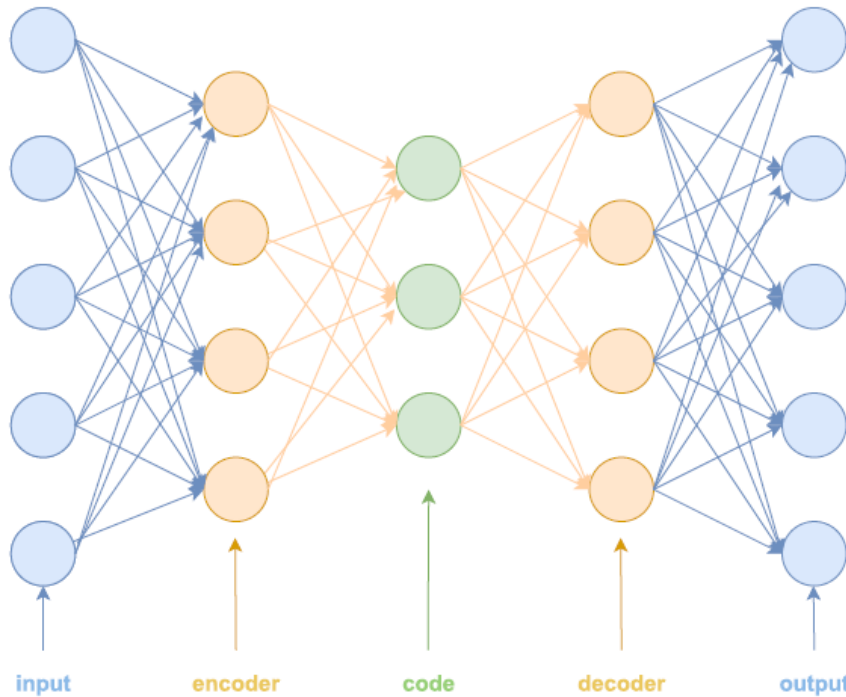


FIG. 3.43: Basic architecture of an AE (Jebril, Pleschberger, and Susto, 2022)

One popular variation of AutoEncoder is Variational Autoencoder (VAE). In VEA,

the Mean and Standard Deviation vectors are inserted as a layer before the latent vector. With this modification, VAE become a Generative Adversarial Network (GAN) because it can generate new content that looks alike original content.

AutoEncoder has different applications, besides anomaly detection and content generation. An example is data denoising. In data denoising application, Under-Complete AutoEncoder is used but original data is corrupted (noise added) before being used as input for training. The expected output is still original data. The objective of the training process become

$$f(g(x_{\text{corrupted}})) = x$$

so that the trained AE will be able to convert corrupted data to clean data.

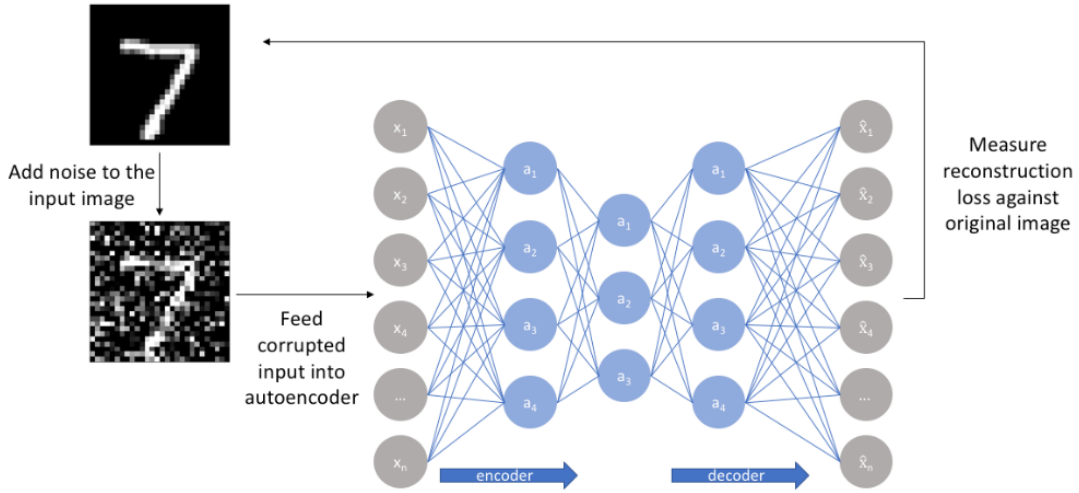


FIG. 3.44: Denoising AutoEncoder (<https://www.jeremyjordan.me/autoencoders/>)

One of most advanced application of AutoEncoder architecture is the Spatio-temporal AutoEncoder for video anomaly detection proposed by Zhao et al. (Zhao et al., 2017b).

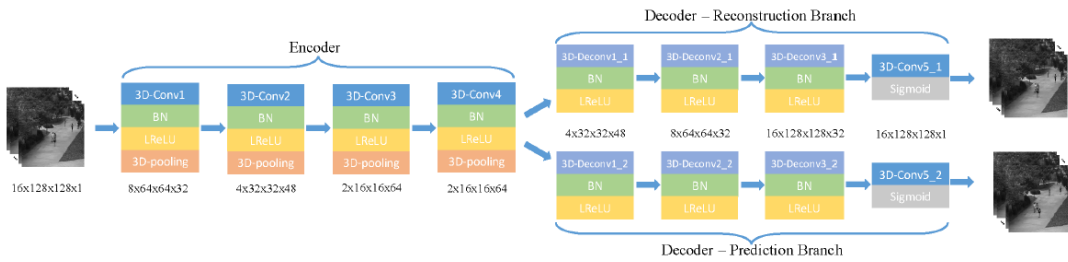


FIG. 3.45: Spatio-temporal AutoEncoder for video anomaly detection (Zhao et al., 2017b)]

AutoEncoder can be used in replacement of dimensional reduction algorithms because they share the same principle: reduce dimensions of data with minimum information loss. Instead of original x , latent representation z can be used as input for other applications such as classification (Fig. 3.46).

The terminology AutoEncoder (AE) or Deep AutoEncoder (DAE) often refers to the Under-Complete AutoEncoder. For Anomaly Detection, only UnderComplete AutoEncoder is needed because it tries to reconstruct exactly the input.

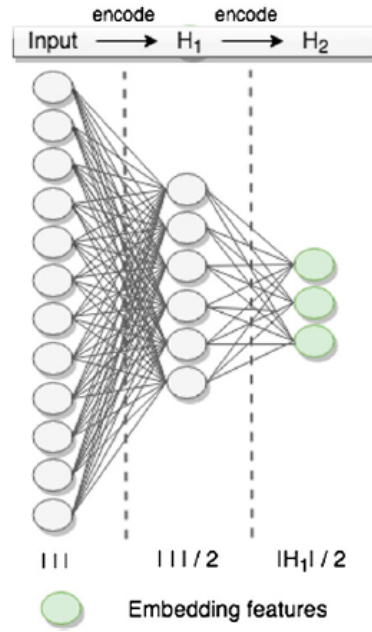


FIG. 3.46: Feature extraction using the encoding function of AE (Corizzo, Ceci, and Japkowicz, 2019)

UnderComplete AutoEncoder can be implemented using standard FFNN or more complicated Deep Learning models such as CNN and LSTM.

LSTM is powerful for predictive models of time-dependent sequential data. If machine data has time dependency (e.g. machine running in cycles like 4-stroke engine or continuous production lines), LSTM can be used for implementation of AutoEncoder.

Dallapiccola implements an AutoEncoder using LSTM for time-dependent data (Dallapiccola, 2020). This model is designed for predictive maintenance of centrifugal pumps. Failure is predictive by Reconstruction Error of n multivariate data samples collected by 80 sensors put on the monitor pumps (Fig. 3.47).

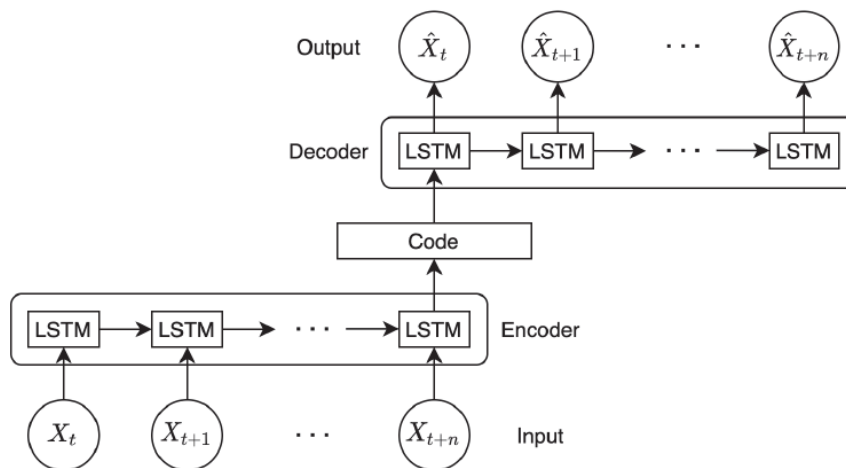


FIG. 3.47: LSTM AutoEncoder for Centrifugal Pumps predictive maintenance (Dallapiccola, 2020)]

In case time dependence is not clear and the FFNN is not powerful enough, CNN can be used. CNN is a variation of Feed Forward Neural Network specialized for

image classification. The idea is to split the image to smaller part for recognition. For example: a human face must have two eyes above nose and mouth. Fig. 3.48 is the vehicle image classification implemented by CNN.

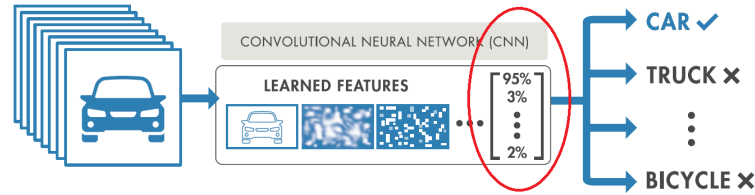


FIG. 3.48: An image classifier implemented by CNN (source: Math-Works)

Yuan et al. implement an AutoEncoder using 2D CNN for predictive maintenance also by monitoring Reconstruction Error (Yuan, Wang, and Wang, 2017). 2D input data is multi-feature sequence data. The first dimension is the list of sensor and the second dimension is time dimension.

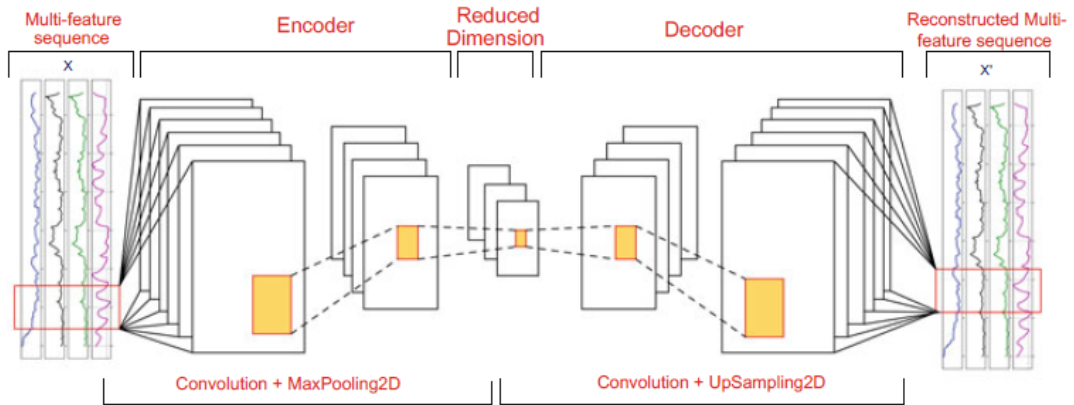


FIG. 3.49: Anomaly Detection AutoEncoder implemented by 2D CNN (Yuan, Wang, and Wang, 2017)]

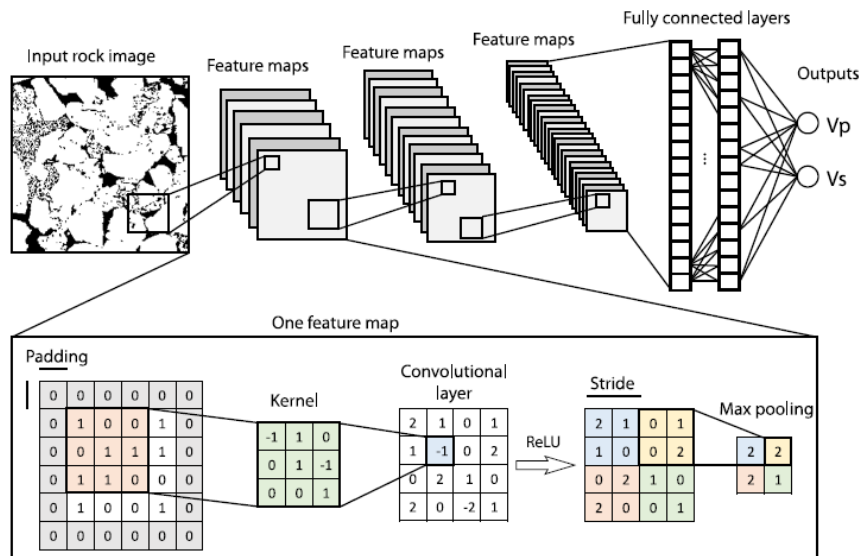


FIG. 3.50: Schematic illustration of a convolutional neural network (Karimpouli and Tahmasebi, 2019)

Fig. 3.50 is an illustration of the way a CNN extracts image hidden feature for classification. CNNs are based on a large number of convolutional layers (i.e., features maps) obtained from convolution of image by many small size kernels. These huge numbers of kernels extract a vast number of features that are hidden in the image (Karimpouli and Tahmasebi, 2019).

The ability to extract hidden features of CNN can be utilized for implementation of AutoEncoder. Combining them to the AutoEncoder architecture, we have the structure as Fig. 3.51. Each layer of the AutoEncoder is a pair of a Convolutional layer and a Pooling layer. This Deep Learning architecture is called Convolutional Deep AutoEncoder (CDAE).

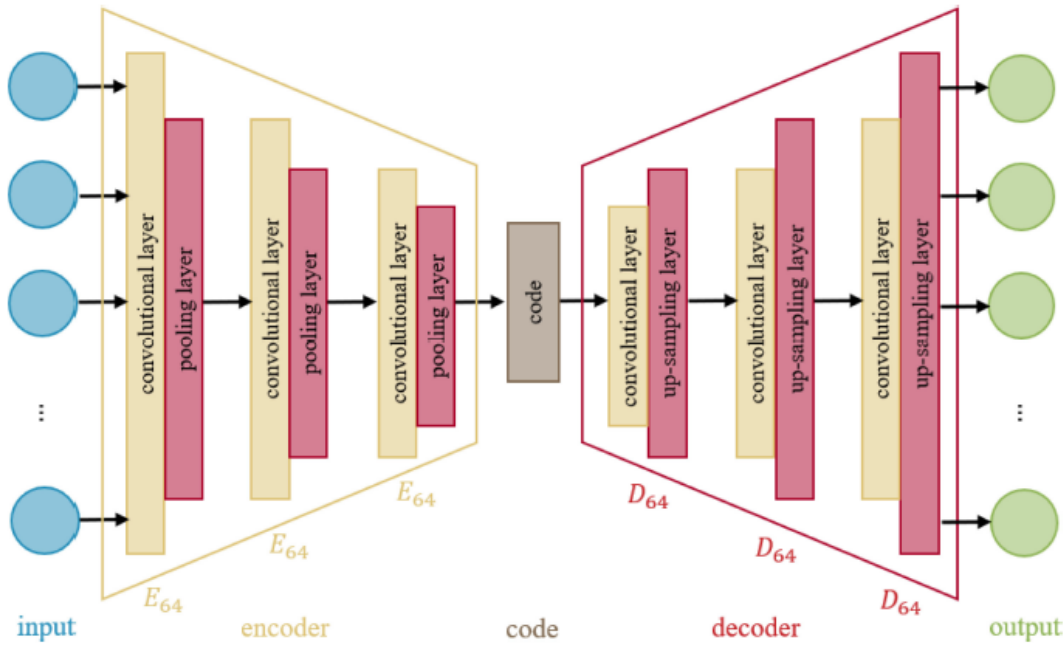


FIG. 3.51: Convolutional Deep AutoEncoder (Jebril, Pleschberger, and Susto, 2022)

Convolutional Deep AutoEncoder (CDAE) is chosen for implementation of the proposed solution of this work. In the rest of this chapter, more details of the way how AutoEncoder can be used for anomaly detection is presented, together with introduction of some studies using this method.

3.4.3 AutoEncoder-based Anomaly Detection method

The AutoEncoder-based anomaly detection method relies on the fact that AutoEncoder has a very high ability in remembering and comparing patterns. After the training process, the trained AutoEncoder will be used to create reconstruction error of new data in order to calculate the anomaly score. Data with high reconstruction error score are considered as anomalous data because only normal data are used to train the AutoEncoder. The AutoEncoder, by correctly fitting undamaged data, can reconstruct normal data very well, while it can not do so with anomalous data that it has not encountered (Pollak et al., 2021). The ability to reconstruct training data of the AutoEncoder is illustrated in Fig. 3.52.

Fig. 3.53 the general flow of anomaly detection approach using AutoEncoder.

Step 1 - Collect data during normal condition: First, a training dataset of normal operating condition of machines need to be prepared. The most important thing of

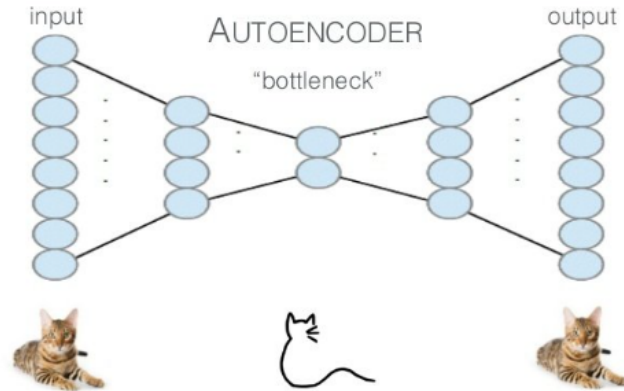


FIG. 3.52: Latent representations in an autoencoder (<http://www.cs.us.es/~fsancho/?e=232>)

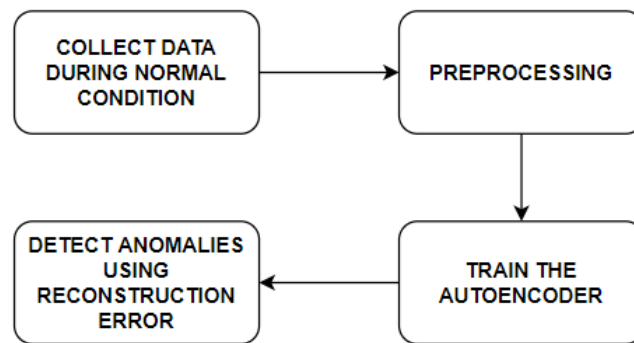


FIG. 3.53: AutoEncoder-based anomaly detection approach (Tedesco et al., 2021)

this step is to make sure that only data of normal conditions are collected. Data of testing, maintenance, set up, special time (e.g. there is a storm), wireless sensor network or SCADA error must be excluded.

Step 2 - Preprocessing: Second step is data preprocessing. For an effective anomaly detection solution, the AutoEncoder training dataset must be as cleaned as possible. It should be cleansed from noise and especially anomalies through an extensive preprocessing step. If this preprocessing step is not effective, the AutoEncoder will learn also anomalous data, and obviously in future it will recognize anomalous data as normal data.

Preprocessing step includes but not limited these tasks: remove constant and noisy signals, drop signals with high empty data gap, change all signal to the same frequency, data imputation (deal with empty value), data scaling and so on.

Tedesco21 et al. also suggest to remove high correlated columns (Tedesco et al., 2021). However, we are not really convinced with that idea. The reason and anti-example will be presented in Chapter 3.

Step 3 - Train the AutoEncoder: Once data are already preprocessed, these data will be used for training the AutoEncoder. The input and output size of AutoEncoder depends on how many signals are kept for anomaly detection.

In principle, a model is used dedicatedly for only one machine, although the machine can be a complex machine (for example: the real case study of Wind Turbine presented in Chapter ??). Exceptions can be simple devices such as light or fan. However, in general it should be avoided. Two machines of the same type, same configuration, at the same industrial site... does not mean that data of this machine

can be used to predict failures of the other, nor their data can be mixed. Machines of same type can have the same training dataset structure and same AutoEncoder structure for simplicity, but not the same trained model.

Step 4 - Detect Anomalies using Reconstruction Error: Once the AutoEncoder is trained, it can be used for anomaly detection by comparing Reconstruction Error of new operating data with Reconstruction Data of training data. Following is exactly what have to be done.

First, we have the training dataset X_{normal} of the machine. It will be used to train the AutoEncoder. Once the training process is converged, we have the trained AutoEncoder.

Then the trained AutoEncoder will be used to reconstruct the training dataset X_{normal} . The output will be X'_{normal} .

Then the Reconstruction Error (RE) of training dataset, call RE_{normal} , will be calculated, using the loss function L (e.g. MSE):

$$RE_{normal} = L(X_{normal}, X'_{normal})$$

We use the RE_{normal} as the baseline for comparison with Reconstruction Error of new data.

Until this step, we have 2 results: the trained AutoEncoder and the Reconstruction Error set of normal data. These two results can be deployed for anomaly detection.

New data of the monitored machine during a time range (e.g. hours, days, weeks, months...), called X_{new} , will be preprocessed in the same way as the training dataset. Then the AutoEncoder will be used to reconstruct this dataset. The output will be X'_{new} . Then the Reconstruction Error of the new dataset will be calculated with the same way:

$$RE_{new} = L(X_{new}, X'_{new})$$

At this step, we have on hand two Reconstruction Error sets of normal data and new data. The mission is to compare them in a certain way (mostly by a statistical/mathematical methods) in order to decide if there are anomalies in new machine data.

There are many advanced analytics techniques for analyzing the result. However, one simple ways is to display the two Reconstruction Error sets in histogram (Fig. 3.54).

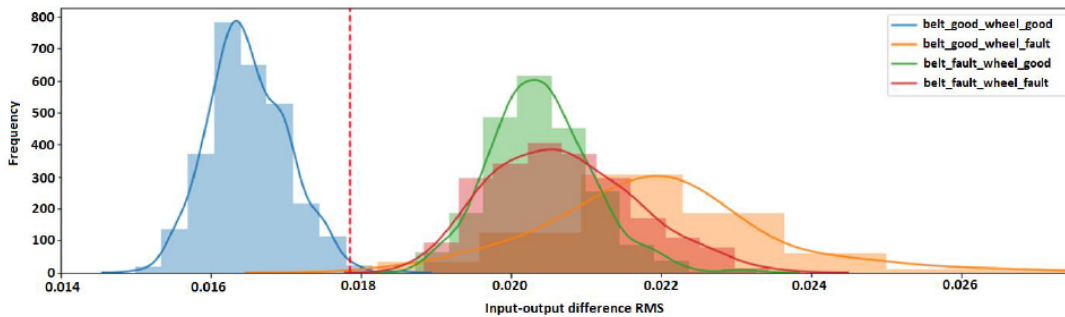


FIG. 3.54: Distribution of reconstruction error for each damage-related class for vertically positioned sensor (Pollak et al., 2021)

Fig. 3.54 is the result of the study by Pollak21 et al., also using AutoEncoder-based method for detecting anomalies of belt drive (Pollak et al., 2021). The vertical dotted line is the Reconstruction Error threshold (RE Threshold) so that only a small

percentage of training data has Reconstruction Error (RE) higher than this threshold. This targeted percentage is usually small, about 3%-7%, and can be called Anomaly Threshold. If RE of a new data sample is higher than the RE Threshold, it will be considered as anomalous data. % anomalous data of new data during a continuous time range will be monitored and compared to the Anomaly Threshold.

Because Fig. 3.54 are simulated failures, most of new data samples exceed the RE threshold. In real cases, if about 10%-15% of new data has RE above the RE threshold, an alarm can be raised.

In summary, there are two thresholds. The Reconstruction Error threshold is used to determine if a data sample is anomalous (if its RE is higher than this RE threshold). The Anomaly Threshold defines how much anomalous data in a continuous monitoring time range can be considered as anomaly.

If these thresholds are set too strict, the AutoEncoder can generate too many false alarms. However, if they are too easy, we may not be able to predict some failures. The choice depends on the opinions of stakeholders and how critical the machine is. For example, for SCADA system, false alarms are preferred than undetected anomalies (Nazir, Patel, and Patel, 2021). In the case study presented in Chapter ??, wind turbines are on the sea, so verifying a false alarm can be easier than coming to the wind power plant to fix a failure.

Compared to the regression method, the Anomaly Detection method using AutoEncoder is much easier to deploy. It is useful to have machine expertise but it's not mandatory. We only need to collect signals related to machine status for learning. These data can be read through I/O module of the machine or we can install sensors around the machine in order to collect machine condition indicators such as vibration, temperature, pressure, noise... For example, in the study by Pollak et al., they simply put vibration sensors on the surface of the belt drive (Pollak et al., 2021).

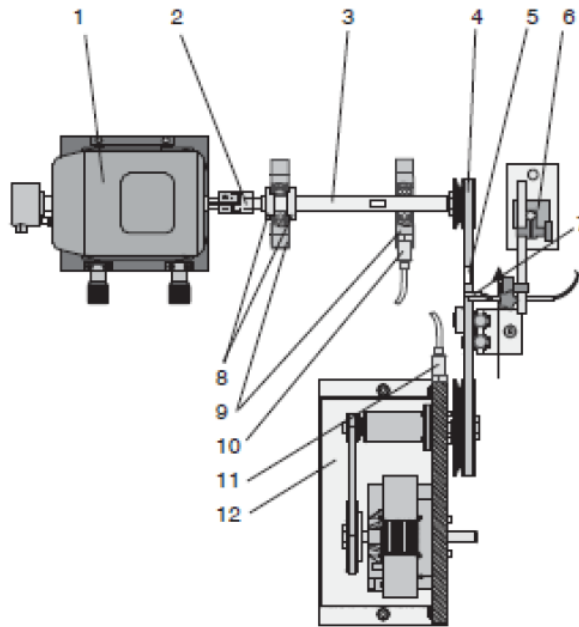


FIG. 3.55: Setup for simulation of vibrations from belt drive (Pollak et al., 2021)

They conduct two experiments: one with sensors installed vertically and one with sensors installed horizontally. Fig. 3.54 is the result of vertically positioned

sensor experiment while Fig. 3.56 is result of horizontally positioned sensor experiment. Both manners of sensor installation show positive results. In this study, only vibration sensors are used. If there are sensors of different types, the ability to detect anomalies will be much more powerful.

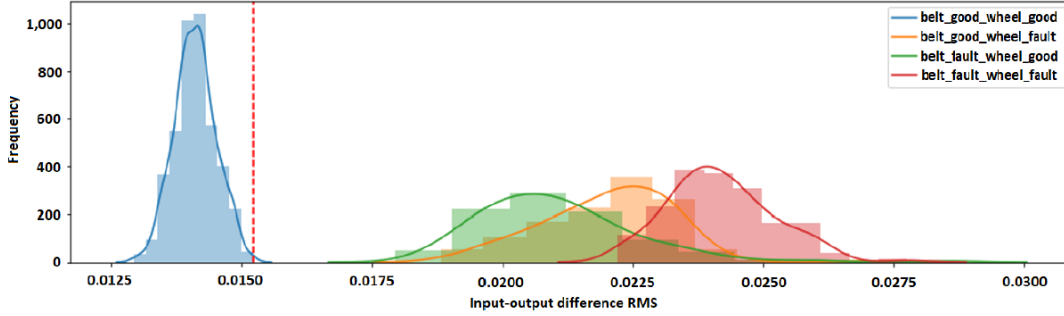


FIG. 3.56: Distribution of reconstruction error for each damage-related class for horizontally positioned sensor (Pollak et al., 2021)

Easy to deploy is the advantage of the Anomaly Detection method using AutoEncoder. However, it has its own challenges. If these challenges are not resolved, the solution cannot be used efficiently in real life.

The first difficulty is to set the anomaly threshold. We need to balance between false alarm and undetected failures, especially if the solution is deployed for a big number of machines. Alfeo et al. try to automate the determination of this threshold so that the anomaly detection solution will have better adaptation capability (Alfeo et al., 2020).

The second challenge is the false alarm caused by the phenomenon of Model Drift (or Model Degradation). It is the scenario that the model trained by obsolete data cannot process new data correctly (Bayram, Ahmed, and Kassler, 2022).

AutoEncoder-base anomaly detection method is based on data Reconstruction Error concept. If the AutoEncoder cannot reconstruct new data, it only means that the new data is not similar to any pattern of the training data. It is not possible to conclude that the new data is generated by a potential failure. Maybe, the new data is simply data of new normal operating conditions. In this case, false alarms are generated by the Model drift (Fig. 3.57).

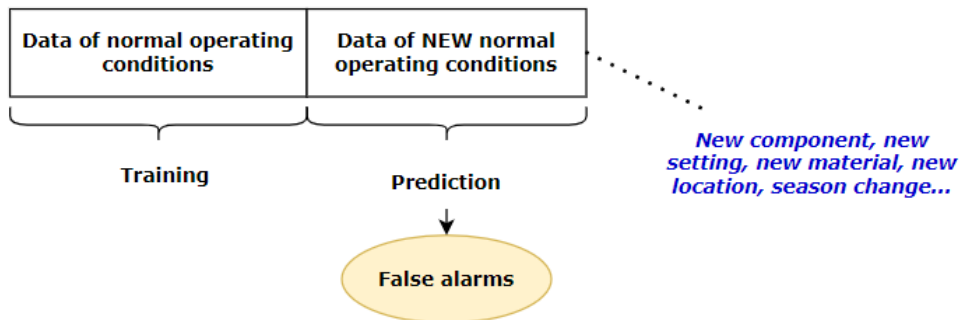


FIG. 3.57: False alarms generated by Model Drift

In other applications such as image classification, in order to avoid model drift, we can simply re-train the model with newer data, the more frequent training the better. However, in the AutoEncoder-base anomaly detection solution, re-training the AutoEncoder with too "new" data have the risk that the AutoEncoder can learn data of potential failures. Studies show that before a failure happens, its root cause

could start months ago and anomalous data was increasing gradually (Bengtsson and Lundström, 2018).

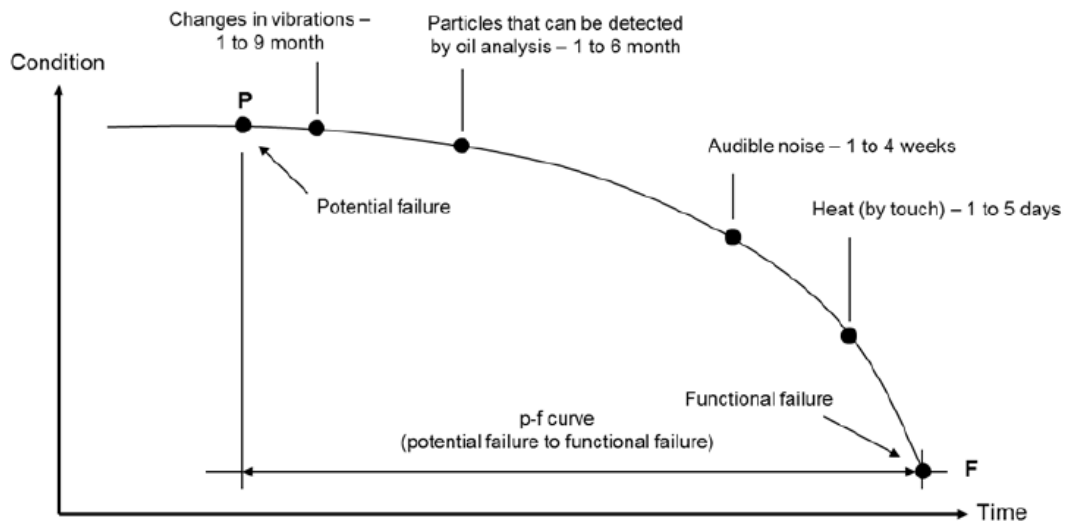


FIG. 3.58: Example of a p-f curve of a ball bearing (Bengtsson and Lundström, 2018)

When the root cause (or the degradation) started, anomalous data was not enough for being detected. If the AutoEncoder was re-trained with that anomalous data, the AutoEncoder will never be able to detect this failure or the failure will be detected too late for avoiding (Fig. 3.59).

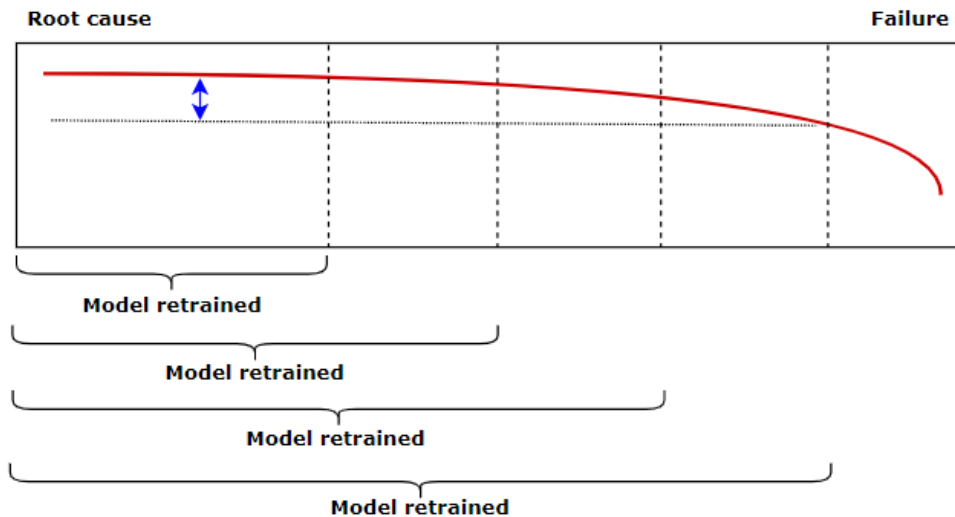


FIG. 3.59: Risk of AutoEncoder being trained by failure data

These difficulties are rarely met when testing the solution with experimental dataset or simulated failures, but when deploying the anomaly detection solution for real cases, the dilemma between risk of false alarm and risk of undetected failures needs a strategy to balance. The decision needs the involvement of real users of the Predictive Maintenance solution. Although at the first look, false alarms seem to be less serious, in long term users can ignore alarms raised by the solution and finally not use the solution anymore.

Despite these challenges, AutoEncoder-based Anomaly Detection method is a very successful technique for the Predictive Maintenance problem. Many studies justify its capability of detecting anomalies of complex industrial machines.

AutoEncoder-based Anomaly Detection method is not an unsupervised learning method, like clustering models which can work with all collected data. It is the semi-supervised learning method because it needs training data of only good working conditions, like for a binary classification mode. However, compared to supervised learning that need labeled and balanced data, training data requirement for AutoEncoder-based Anomaly Detection method is much easier. Therefore, this approach has high feasibility for deployment in real life. Many studies of Anomaly Detection using this approach can be tested with real case studies (Roelofs et al., 2021) (Alfeo et al., 2020) (Serradilla et al., 2021) (Lutz et al., 2020) (Tedesco et al., 2021).

3.4.4 Referenced studies of Anomaly Detection using AutoEncoder

A scalable deep learning-based approach for anomaly detection in semiconductor manufacturing (Tedesco et al., 2021)

Tedesco et al. propose a general and scalable AutoEncoder-based approach for anomaly detection in semiconductor manufacturing, a very complex manufacturing industry. Primary steps of their approach are already presented in 3.4.3 (Fig. 3.53).

The main strength of this approach is its generality and scalability because it is not based on specific characteristics of any dataset, machines or industrial process. Therefore, it can be used to handle heterogeneous datasets come from different equipment, work centers and even production sites. They consider the Anomaly Detection problem as multivariate problem without time dependency. They not only propose an approach but also describe details of their implementation with recommended best practices.

First, in data processing step, they propose to remove constant (or N/A) columns, highly correlated columns and noisy columns of the training dataset. Noise features can be removed by the Kolmogorov-Smirnov (KS) test presented by Andrey Kolmogorov and Nikolai Smirnov in 1933. Then the training dataset must be scaled. This step is also called Data Normalization so that all features belong to the same value range, otherwise the calculation of the distance will not be correct.

They also shuffle randomly the training dataset in order to remove any possible time dependence in the data. The motivation of this step is that except data is clearly known as time-dependent and a time dependent approach is chosen, they don't want the AutoEncoder to learn any time dependencies. Each training data sample (or a continuous block of data sample called data window) will be learnt by Deep Learning model as a separate snapshot of the machine condition.

They also recommend an initial hyperparameter set which can start with most of AutoEncoder models. First, the percentage used in splitting process is 90% (90% for the training set and 10% for the test set). The default loss function used for training and evaluation can be Mean Squared Error (MSE). The optimizer can be Adam optimizer (Adaptive Moment Estimation, an improvement of Stochastic Gradient Descent (SGD) algorithm). Early Stopping strategy can be used to perform regularization and avoid overfitting. Although ReLU activation function is often used for AutoEncoder, they suggest to use LeakyReLU in case the dying ReLU problem is met.

About the structure of the AutoEncoder, they proposed a Tournament Search Architecture Optimization algorithm to find the best depth and size of the AutoEncoder in respect of each input dataset. They use Statistical Process Control (SPC) techniques in order to decide if a data point (data sample) is considered as anomalous. Key elements of this technique are Upper Control Limit (UCL) and a Lower Control Limit (LCL). If a data sample having Reconstruction Error within these limits, it is an inlier (normal data). Otherwise, it is an outlier (anomaly). In this context, only Upper Control Limit is used. It is called Anomaly Threshold and defined based on Reconstruction Error of the training dataset. Reconstruction Error of each new data sample is called Anomaly Score and can be monitored using Control Chart. More applications of Control Chart for Anomaly Detection in Manufacturing can be found in this book (Tran, 2022).

Finally, in terms of performance, this approach outperforms compared to Isolation Forests (iForests), introduced by Liu et al. 2008, and Local Outlier Factor, an anomaly score introduced by Kriegel et al. in 2009.

Evaluation of anomaly detection of an autoencoder based on maintenance information and SCADA-data (Lutz et al., 2020)

Similar to Tedesco et al., Lutz et al. describe and evaluate in details their Anomaly Detection method using AutoEncoder to reconstruct SCADA data (Lutz et al., 2020). Their case studies are Wind Turbines of a Wind Farm in German North Sea. Data is sensor data acquired by SCADA. Wind Turbines are very complex machines which are expected to run continuously and intensively. Therefore, if their solution can be deployed for this type of machine, it can be considered as a universal approach for many other types of machine (like Tedesco et al.). Only content not mentioned by Tedesco et al. is listed here.

Like most of AutoEncoder-base anomaly detection method, only data of normal working condition is selected for training. In this case, they are Production, Ready, Run up and High wind shutdown. All of other operational modes such as Failure, Manual Stop, Service... are excluded.

Then in data preprocessing step, only sensors with at least 80% of values are integer or float value are selected. Otherwise, the sensor is dropped.

Besides data scaling, they suggest that for Data Imputation (deal with missing value), linear interpolation should not be used. Instead, the mean value of all other values of the sensor should be used to replace missing values. Their logic is that if there is data gap, data gap often appears over a period of several days, so interpolation based on the values before and after the gap is not appropriate.

Their AutoEncoder structure is quite simple. It is a minimum UnderComplete AutoEncoder with three layers. Input and output layers have 1350 neurons, while the bottle neck layer has 50 neurons. Instead of Leaky ReLU activation function, they use the ReLU activation function. The number of epochs for training is only 10.

However, for Reconstruction Error measure, they use a complicated distance. Instead of MAE, MSE or RMSE which implicitly assume that all sensor values are independent, they use the Mahalanobis distance with the assumption that sensor values are dependent on one another.

With these settings, their model produces positive result. Instead of comparing to other methods, they use real service reports to verify their model by checking if it can detect failures recorded in service reports. A similar verification method is chosen in this work.

Autoencoder-based anomaly root cause analysis for wind turbines (Roelofs et al., 2021)

The study of Roelofs et al. is an application of AI to energy industry. This research has two real world case studies. The first case study is also the Wind Farm dataset used in this work (Inovação, 2018). The second case study is the SCADA dataset of also a wind power plant located in German North Sea.

Their AutoEncoder-base anomaly detection method is similar to other studies. However, they have the ambition to create an interpretable model.

First, they try to analyze elementwise Reconstruction Error of each signal as its contribution to the whole Reconstruction Error. Fig. 3.60 is the heat map of reconstruction error of features (sensor signal) after a simulated error of one signal is added to good condition data (Fig. 3.61).

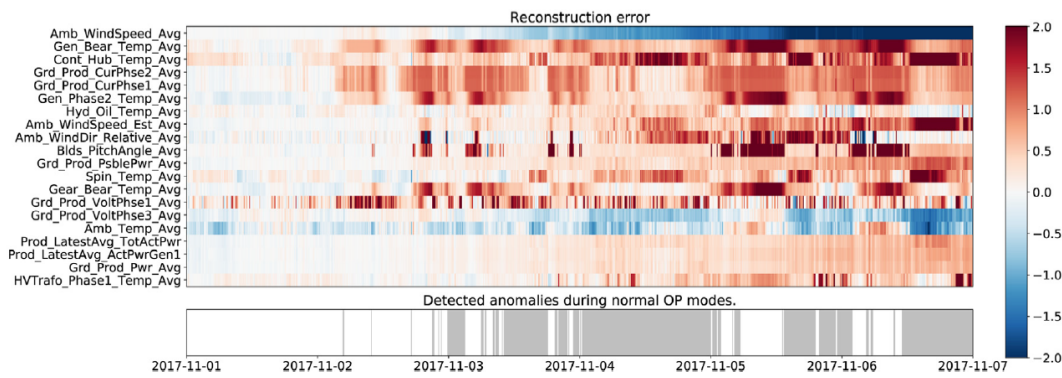


FIG. 3.60: Heat map of the scaled reconstruction error for the prediction period with added wind speed sensor error (Roelofs et al., 2021)

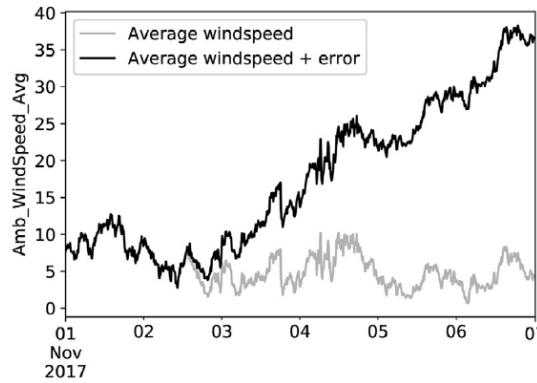


FIG. 3.61: Simulated error added to good condition data (Roelofs et al., 2021)

However, this analysis cannot isolate the root cause of the anomaly because the fully connected structure of neural network allows the error to be propagated and affect all outputs (Fig. 3.62).

To solve this problem, they propose and implement the ARCANA (Anomaly Root Cause Analysis) algorithm to find which inputs contributes to the anomaly detected by the AutoEncoder. Fig. 3.63 shows that the ARCANA algorithm can recognize which abnormal signal can be the root cause of the anomaly.

Although explainable AI is not in the scope of this work, from their result, we know that analyzing Reconstruction Error of each signal cannot help to discover the

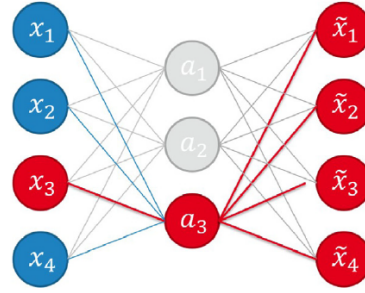


FIG. 3.62: Error is propagated through the network (Roelofs et al., 2021)

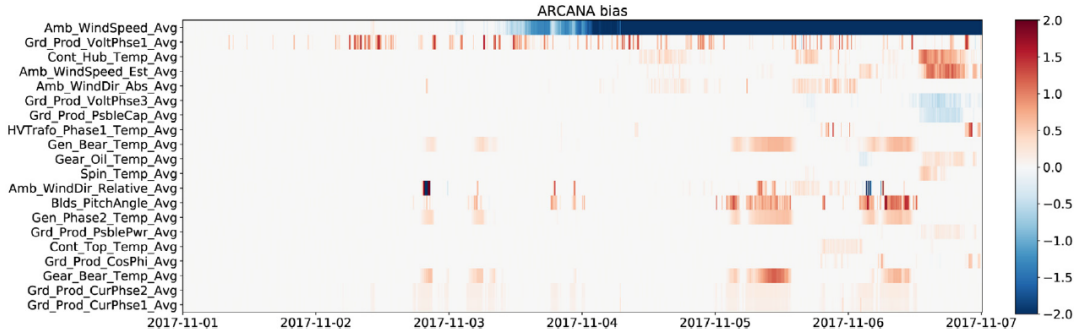


FIG. 3.63: Heat map after applied the ARCANA algorithm (Roelofs et al., 2021)

root cause of the anomaly. Therefore, this function will not be implemented in the software.

Using an autoencoder in the design of an anomaly detector for smart manufacturing (Alfeo et al., 2020)

The Anomaly Detection solution by Alfeo et al. aims at supporting smart manufacturing operators with no data science background. The development of anomaly detection solutions in real-world manufacturing demands a trade-off between system management cost and detection accuracy. It depends on the anomaly threshold which distinguishes normal data and anomalous data. Restrictive thresholds may increase the false negatives, while loose thresholds result in many false positives. In the machine learning pipeline with the involvement of three roles (Data Engineer, Data Scientist and Business Expert), the Data Scientist is in charge of the Anomaly Threshold setup (Fig. 3.64).

In order to improve process performance, they develop a Discriminator that can adjust the anomaly score threshold automatically using heuristic-based algorithms (Fig. 3.65).

Their case study is the washing process of industry laundry. They compare their results with Isolation Forest, the current best-in-class Anomaly Detection algorithm.

Although automating the Anomaly Threshold is not in the scope of this work, their Anomaly Detection pipeline makes clear the user roles and the necessary collaboration between them for anomaly detection, which is rarely discussed in researches of this topic.

Their pipeline also inspires us to define user roles (Machine Operator, Machine Expert) of the Predictive Maintenance software so that the AutoEncoder can be

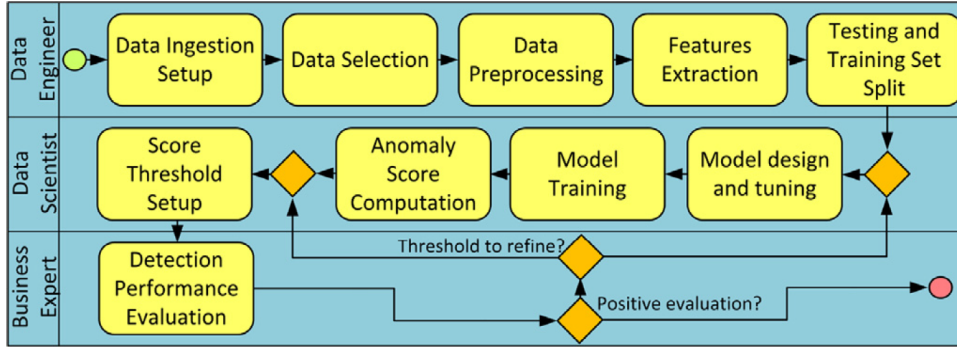


FIG. 3.64: Anomaly detection pipeline (Alfeo et al., 2020)

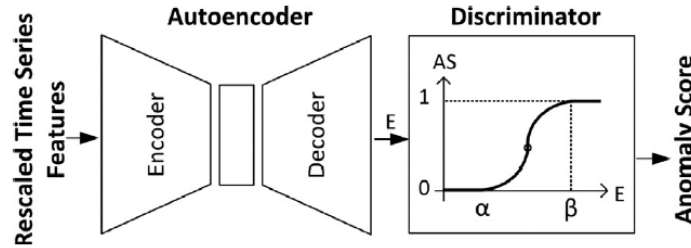


FIG. 3.65: Discriminator for Anomaly Score (Alfeo et al., 2020)

trained and re-trained efficiently with right data. The Data Scientist role in the organization also inspires us to create an open mechanism so that Data Scientist users can replace our built-in AutoEncoder model by their own models.

3.4.5 Conclusion

In conclusion, after reviewing several state-of-the-art studies of Predictive Maintenance, especially studies of Anomaly Detection, the AutoEncoder-based Anomaly Detection method is chosen for the proposed Predictive Maintenance system implemented in this work.

The Convolutional Deep AutoEncoder (CDAE) implemented by Convolutional Neural Network (CNN) is chosen as the built-in AutoEncoder for monitoring data Reconstruction Error, although it is open for Data Scientist users to import their own AutoEncoder models.

The predictive model should be as general and scalable as possible, not based on characteristics of specific dataset or machines, requiring minimum detailed knowledge of monitored machines, although machine expertise will be helpful. This is the objective and also the ambition of this work.

Chapter 4

Proposed Solution: description of the proposed solution

4.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

4.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

4.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

4.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi

tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 5

Implemented Solution: detailed implementation of the proposed solution

5.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

5.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

5.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

5.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in.

Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 6

Case Study 1

6.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

6.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

6.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

6.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 7

Case Study 2

7.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

7.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

7.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

7.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Chapter 8

Conclusions and Future Works

8.1 Main Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam ultricies lacinia euismod. Nam tempus risus in dolor rhoncus in interdum enim tincidunt. Donec vel nunc neque. In condimentum ullamcorper quam non consequat. Fusce sagittis tempor feugiat. Fusce magna erat, molestie eu convallis ut, tempus sed arcu. Quisque molestie, ante a tincidunt ullamcorper, sapien enim dignissim lacus, in semper nibh erat lobortis purus. Integer dapibus ligula ac risus convallis pellentesque.

8.1.1 Subsection 1

Nunc posuere quam at lectus tristique eu ultrices augue venenatis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam erat volutpat. Vivamus sodales tortor eget quam adipiscing in vulputate ante ullamcorper. Sed eros ante, lacinia et sollicitudin et, aliquam sit amet augue. In hac habitasse platea dictumst.

8.1.2 Subsection 2

Morbi rutrum odio eget arcu adipiscing sodales. Aenean et purus a est pulvinar pellentesque. Cras in elit neque, quis varius elit. Phasellus fringilla, nibh eu tempus venenatis, dolor elit posuere quam, quis adipiscing urna leo nec orci. Sed nec nulla auctor odio aliquet consequat. Ut nec nulla in ante ullamcorper aliquam at sed dolor. Phasellus fermentum magna in augue gravida cursus. Cras sed pretium lorem. Pellentesque eget ornare odio. Proin accumsan, massa viverra cursus pharetra, ipsum nisi lobortis velit, a malesuada dolor lorem eu neque.

8.2 Main Section 2

Sed ullamcorper quam eu nisl interdum at interdum enim egestas. Aliquam placerat justo sed lectus lobortis ut porta nisl porttitor. Vestibulum mi dolor, lacinia molestie gravida at, tempus vitae ligula. Donec eget quam sapien, in viverra eros. Donec pellentesque justo a massa fringilla non vestibulum metus vestibulum. Vestibulum in orci quis felis tempor lacinia. Vivamus ornare ultrices facilisis. Ut hendrerit volutpat vulputate. Morbi condimentum venenatis augue, id porta ipsum vulputate in. Curabitur luctus tempus justo. Vestibulum risus lectus, adipiscing nec condimentum quis, condimentum nec nisl. Aliquam dictum sagittis velit sed iaculis. Morbi tristique augue sit amet nulla pulvinar id facilisis ligula mollis. Nam elit libero, tincidunt ut aliquam at, molestie in quam. Aenean rhoncus vehicula hendrerit.

Bibliography

- Aggarwal, Charu C et al. (2018). "Neural networks and deep learning". In: *Springer* 10, pp. 978–3.
- Alfeo, Antonio L et al. (2020). "Using an autoencoder in the design of an anomaly detector for smart manufacturing". In: *Pattern Recognition Letters* 136, pp. 272–278.
- Bayram, Firas, Bestoun S Ahmed, and Andreas Kassler (2022). "From concept drift to model degradation: An overview on performance-aware drift detectors". In: *Knowledge-Based Systems*, p. 108632.
- Bengtsson, Marcus and Gunnar Lundström (2018). "On the importance of combining "the new" with "the old"—One important prerequisite for maintenance in Industry 4.0". In: *Procedia manufacturing* 25, pp. 118–125.
- Cao, Qiushi et al. (2022). "KSPMI: a knowledge-based system for predictive maintenance in industry 4.0". In: *Robotics and Computer-Integrated Manufacturing* 74, p. 102281.
- Corizzo, Roberto, Michelangelo Ceci, and Nathalie Japkowicz (2019). "Anomaly detection and repair for accurate predictions in geo-distributed big data". In: *Big Data Research* 16, pp. 18–35.
- Da Silva, Ivan Nunes et al. (2017). "Artificial neural networks". In: *Cham: Springer International Publishing* 39.
- Dallapiccola, Damiano (2020). "Predictive maintenance of centrifugal pumps: a Neural Network approach". PhD thesis. ETSI_Informatica.
- Fink, Olga (2020). "Data-driven intelligent predictive maintenance of industrial assets". In: *Women in industrial and systems engineering*. Springer, pp. 589–605.
- Hashemian, Hashem M (2010). "State-of-the-art predictive maintenance techniques". In: *IEEE Transactions on Instrumentation and measurement* 60.1, pp. 226–236.
- Huuhtanen, Timo and Alexander Jung (2018). "Predictive maintenance of photovoltaic panels via deep learning". In: *2018 IEEE Data Science Workshop (DSW)*. IEEE, pp. 66–70.
- Inovação, EDP (2018). *EDPR wind farm open data: wind turbine SCADA signals and historical failure logbook from 2016 and 2017*. URL: <https://opendata.edp.com/pages/Windfarms/> (visited on 06/26/2022).
- Jebril, Hana TT, Martin Pleschberger, and Gian Antonio Susto (2022). "An Autoencoder-Based Approach for Fault Detection in Multi-Stage Manufacturing: A Sputter Deposition and Rapid Thermal Processing Case Study". In: *IEEE Transactions on Semiconductor Manufacturing* 35.2, pp. 166–173.
- Karimpouli, Sadegh and Pejman Tahmasebi (2019). "Image-based velocity estimation of rock using convolutional neural networks". In: *Neural Networks* 111, pp. 89–97.
- Khan, Samir and Takehisa Yairi (2018). "A review on the application of deep learning in system health management". In: *Mechanical Systems and Signal Processing* 107, pp. 241–265.

- Lopez, Felipe et al. (2017). "Categorization of anomalies in smart manufacturing systems to support the selection of detection mechanisms". In: *IEEE Robotics and Automation Letters* 2.4, pp. 1885–1892.
- Luo, Tie and Sai G Nagarajan (2018). "Distributed anomaly detection using autoencoder neural networks in WSN for IoT". In: *2018 IEEE International Conference on Communications (ICC)*. IEEE, pp. 1–6.
- Lutz, Marc-Alexander et al. (2020). "Evaluation of anomaly detection of an autoencoder based on maintenance information and SCADA-data". In: *Energies* 13.5, p. 1063.
- Meesublak, Koonlachat and Tosapol Klinsukont (2020). "A cyber-physical system approach for predictive maintenance". In: *2020 IEEE International Conference on Smart Internet of Things (SmartIoT)*. IEEE, pp. 337–341.
- Michelucci, Umberto (2018). "Applied Deep Learning". In: *A Case-Based Approach to Understanding Deep Neural Networks*.
- Microsoft (2022). *Concepts of AI, Machine Learning and Deep Learning*. URL: <https://docs.microsoft.com/en-us/azure/machine-learning/concept-deep-learning-vs-machine-learning> (visited on 06/26/2022).
- Moleda, Marek, Alina Momot, and Dariusz Mrozek (2020). "Predictive maintenance of boiler feed water pumps using SCADA data". In: *Sensors* 20.2, p. 571.
- Nazir, Sajid, Shushma Patel, and Dilip Patel (2021). "Autoencoder based anomaly detection for SCADA networks". In: *International Journal of Artificial Intelligence and Machine Learning (IJAIML)* 11.2, pp. 83–99.
- Pollak, Artur et al. (2021). "Prediction of belt drive faults in case of predictive maintenance in industry 4.0 platform". In: *Applied Sciences* 11.21, p. 10307.
- Rivas, Alberto et al. (2019). "A predictive maintenance model using recurrent neural networks". In: *International workshop on soft computing models in industrial and environmental applications*. Springer, pp. 261–270.
- Roelofs, Cyriana MA et al. (2021). "Autoencoder-based anomaly root cause analysis for wind turbines". In: *Energy and AI* 4, p. 100065.
- Saxena, Abhinav and Kai Goebel (2008). "Turbofan engine degradation simulation data set". In: *NASA Ames Prognostics Data Repository*, pp. 1551–3203.
- Serradilla, Oscar et al. (2021). "Adaptable and explainable predictive maintenance: Semi-supervised deep learning for anomaly detection and diagnosis in press machine data". In: *Applied Sciences* 11.16, p. 7376.
- Susto, Gian Antonio et al. (2014). "Machine learning for predictive maintenance: A multiple classifier approach". In: *IEEE transactions on industrial informatics* 11.3, pp. 812–820.
- Tedesco, Simone et al. (2021). "A scalable deep learning-based approach for anomaly detection in semiconductor manufacturing". In: *2021 Winter Simulation Conference (WSC)*. IEEE, pp. 1–12.
- Tran, Kim Phuc (2022). "Introduction to Control Charts and Machine Learning for Anomaly Detection in Manufacturing". In: *Control Charts and Machine Learning for Anomaly Detection in Manufacturing*. Springer, pp. 1–6.
- Yuan, Jin, Kesheng Wang, and Yi Wang (2017). "Deep learning approach to multiple features sequence analysis in predictive maintenance". In: *International Workshop of Advanced Manufacturing and Automation*. Springer, pp. 581–590.
- Zhang, Weiting, Dong Yang, and Hongchao Wang (2019). "Data-driven methods for predictive maintenance of industrial equipment: A survey". In: *IEEE Systems Journal* 13.3, pp. 2213–2227.

- Zhao, Pushe et al. (2017a). "Advanced correlation-based anomaly detection method for predictive maintenance". In: *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*. IEEE, pp. 78–83.
- Zhao, Yiru et al. (2017b). "Spatio-temporal autoencoder for video anomaly detection". In: *Proceedings of the 25th ACM international conference on Multimedia*, pp. 1933–1941.